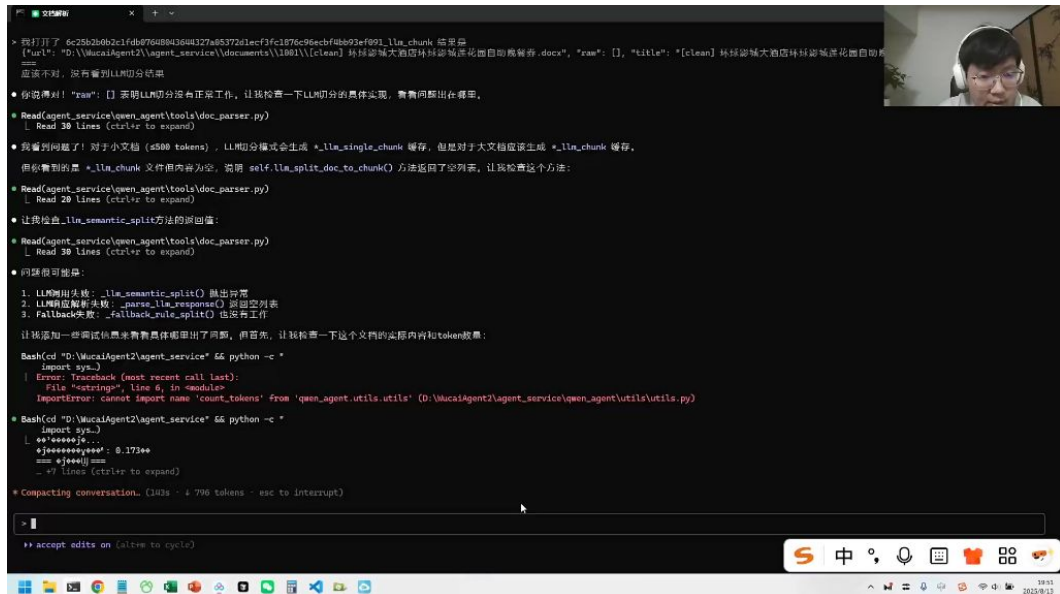


一、文档解析06:28

1. AI agent的火热与项目反馈 06:30

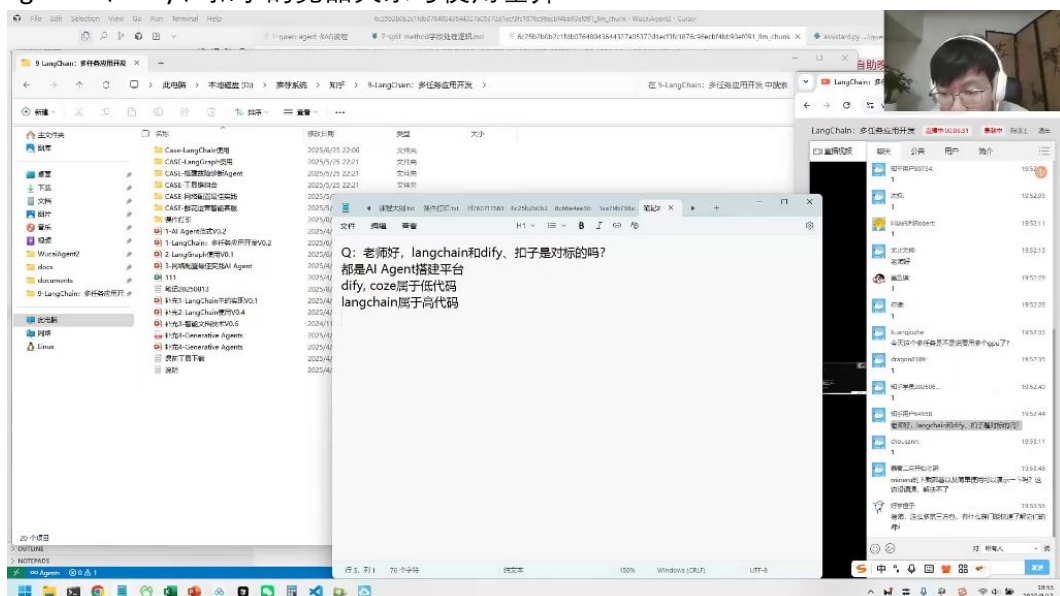


- 市场热度：今年AI agent需求非常火热，很多客户都有类似需求
- 项目反馈：近期收到大量客户反馈，正在处理相关项目需求

2. GPU在多任务中的应用与必要性 07:06

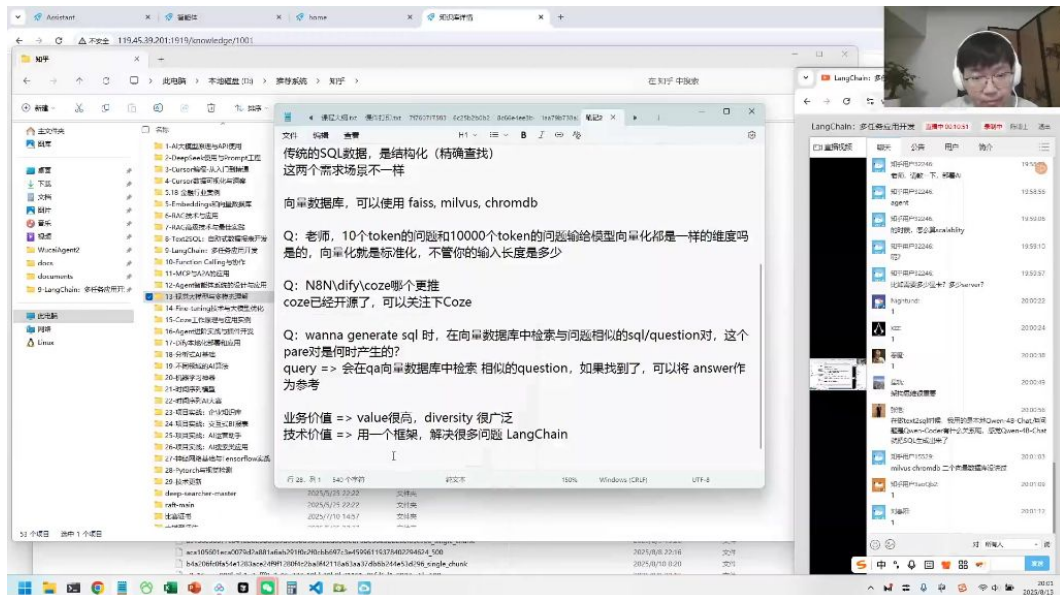
- 使用场景：多任务处理时多个GPU的应用
- 必要性评估：
 - 除非进行微调，否则基本不需要GPU
 - 未来工作中GPU可理解为运维和部署工作
 - 重点应关注基于服务实现的业务场景

3. LangChain、Dify、扣子的竞品关系与使用差异 07:34

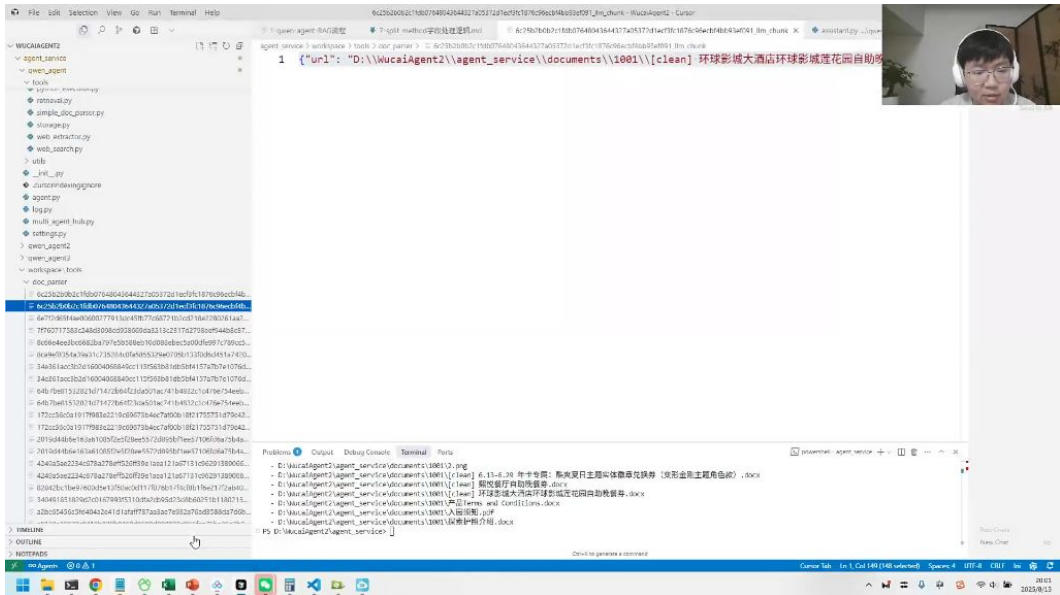


- 竞品关系：三者都是AI agent搭建平台，属于竞品关系
- 使用差异：
 - Dify和扣子属于低代码平台

- LangChain属于高代码平台
 - 选择依据主要取决于使用习惯
4. Mindu的下载部署及简单使用 08:25
- 部署说明：将有专门课程讲解下载部署
 - 性能特点：
 - 本地CPU可运行但速度慢（约20-30分钟）
 - 主要用于网页推送
5. 了解第三方包API的诀窍 09:03
- 最佳实践：
 - 通过AI快速了解API
 - 基于业务需求理解API调用流程
 - AI可在5分钟左右整理好工具使用方法 e.g.cursor
6. 传统SQL数据库与向量数据库的区别 09:56



- 应用场景：
 - 传统SQL：结构化数据，**精确查找**
 - 向量数据库：存储向量，**相似度检索**
 - 代表产品：Faiss、Milvus、ChromDB等
7. 输入token问题与向量化 10:56
- 标准化处理：
 - 无论输入长度（10或1000个token）
 - 向量化后维度相同
 - 信息量差异不影响标准化过程
8. Dify与Coze的对比及推荐 11:28
- 产品对比：
 - Coze更新速度更快
 - Dify维护团队较小
 - Coze商业化场景更丰富
 - 功能差异：
 - Dify将Excel数据视为文本，相似度查找
 - Coze支持精确查找
 - 推荐建议：优先考虑Coze
9. 业务价值与技术价值 14:06



-
- **业务价值：**
 - 价值高
 - 多样性广泛（金融、制造、医疗等多行业）
- **技术价值：**
 - 通过框架解决多类问题
 - 如LangChain可解决多种应用场景

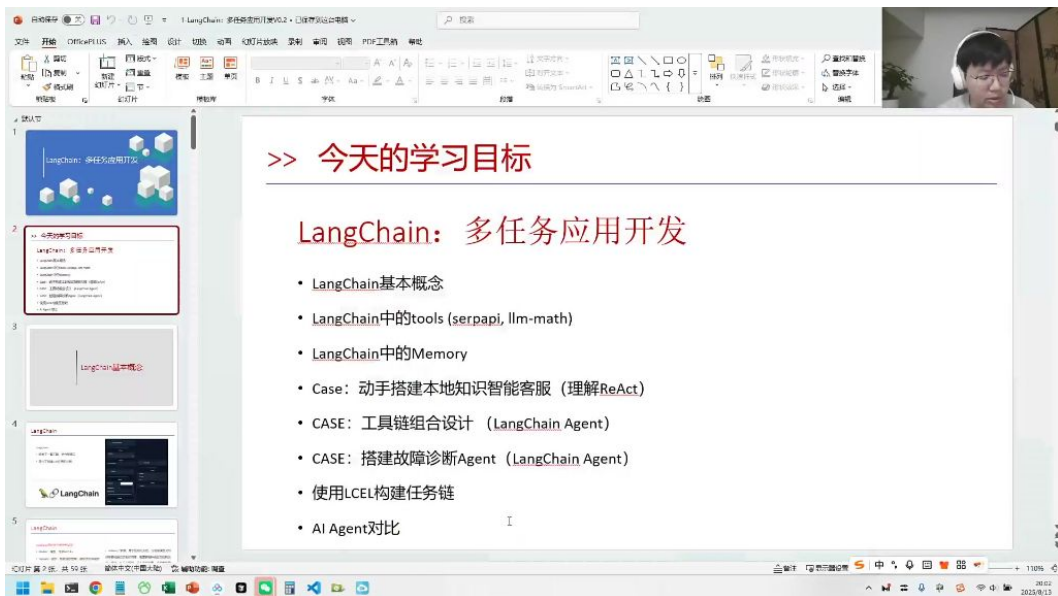
10. 与AI共生的状态 15:35

- **实践体会：**
 - 需要与AI保持共生状态
 - 使用多种工具（如Coze等）
 - 会员服务实际价值需评估

11. 架构思维与逻辑思维的重要性 15:52

- **核心能力：**
 - 逻辑思维比编码能力更重要
 - AI生成的逻辑需要人工验证
 - 错误逻辑可能导致更多时间成本
- **效率评估：**
 - AI节省编码时间
 - 但无法节省逻辑验证时间

12. 逻辑思维的培养与项目实践 16:54



- 培养方法：
 - 通过小型项目实践
 - 理解完整流程（用户提问→处理环节→结论）
 - 思考流程优化和功能扩
- 开发习惯：
 - 简单操作（如文件重命名）手动完成
 - 复杂逻辑交由AI实现

二、多任务应用开发 17:58

1. LangChain基本概念 18:03

workflow 的工作流模式，提供一系列工具，组件和接口
简化创建LLM应用的过程

1) 例:Langchain使用 25:25

- Prompt Template组件
 - 基本用法：在LangChain中使用PromptTemplate组件创建可复用的提示词模板，其中固定内容与变量分离
 - 变量替换：用户只需传入变化部分，如示例中的"colorful socks"替换product变量
 - 工作流示例：
 - 输入："colorful socks" → 拼成完整问句 → 大模型生成结果
 - 输出创意名称如"彩虹rangers tos"等
 - 更换变量为"广告设计"后输出"创世界广汇"等中文名称
- LangChain与LangGraph区别
 - 架构差异：
 - LangChain是**线性结构** ($A \rightarrow B \rightarrow C \rightarrow D$)
 - LangGraph是**图结构** (支持多路径如 $A \rightarrow B/C/E$)
 - 能力对比：
 - $\text{LangGraph} \geq \text{LangChain}$
 - LangGraph适合更复杂的任务流
 - 版本关系：LangGraph是LangChain的升级版，同属一个技术体系

2) 例:工具使用 30:07

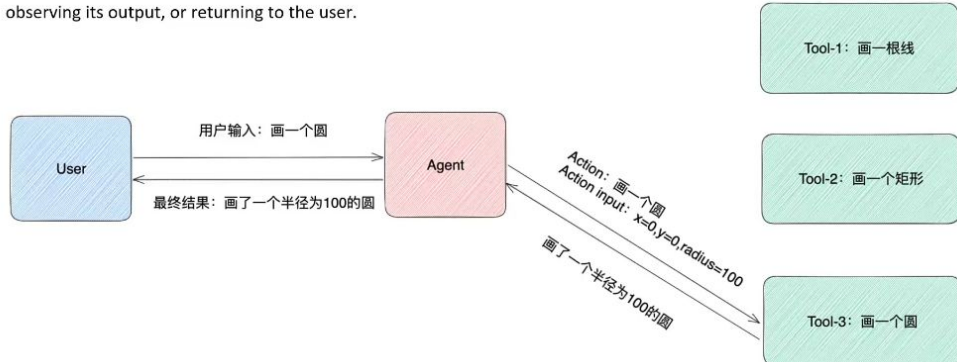
- Agent工作机制

Agent的作用

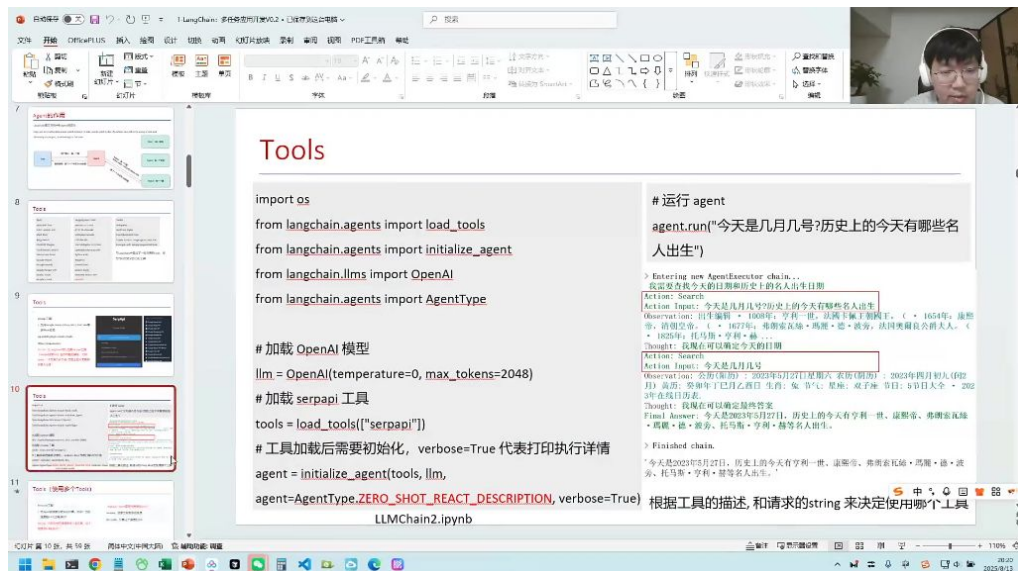


LangChain官方文档中对Agent的定义:

they use an LLM to determine which actions to take and in what order. An action can either be using a tool and observing its output, or returning to the user.



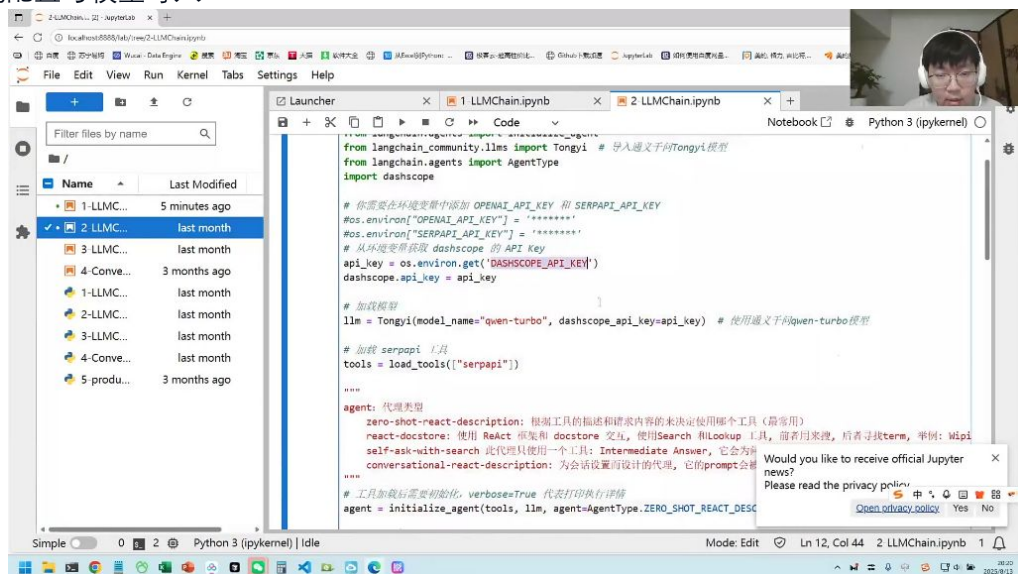
-
- **核心功能:** 通过LLM决策工具调用顺序, 定义见官方文档"determine which actions to take and in what order"
- **工具调用流程:**
 - 用户输入需求 (如"画一个圆")
 - Agent匹配注册工具 (从画线/画矩形/画圆中选择)
 - 解析参数 (圆心 $xy=0,0$, 半径 $r=100$)
 - 执行并返回结果
- **历史地位:** **LangChain最早提出工具调用概念**, 后被各大模型厂商集成
- **工具集扩展**
 - **内置工具:**
 - 搜索引擎 (Serpapi/Google/Baidu)
 - Google Earth/Server API
 - Python脚本执行器
 - **自定义开发:** 支持用户根据业务需求创建专属工具
- 3) Serpapi工具 33:12 <https://serpapi.com/>
- **功能概述**
 - **多平台支持:** 集成Google/Baidu/Yahoo等搜索API
 - **安装方式:** 通过`pip install google-search-results`安装
 - **免费计划:** 提供100次/月非商业用途的免费调用额度
- **实际应用案例**



- 典型查询：解答"今天是什么日期？历史上的今天有哪些名人出生"
- 实现步骤：
 - 加载OpenAI模型（设置temperature=0, max_tokens=2048）
 - 初始化工具：tools = load_tools(["serpapi"])
 - 创建Agent：initialize_agent(..., agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION)
- 执行过程：
 - 自动识别需要搜索工具
 - 返回历史人物信息（如1654年康熙帝出生等）
 - 最终整合输出日期和名人列表

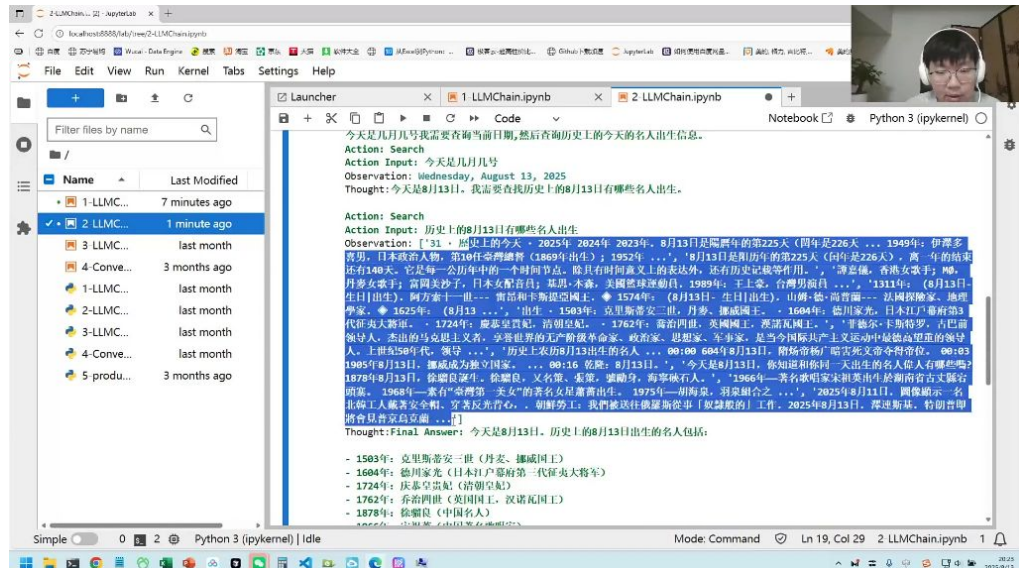
4) 例题: LangChain问答 35:27

- 环境配置与模型导入



- 环境变量设置：需要在系统中添加OPENAI_API_KEY、SERPAPI_API_KEY和DASHSCOPE_API_KEY等密钥，Windows系统可通过搜索"环境变量"进行设置
- 模型导入：使用from langchain_community.llms import Tongyi导入通义千问模型，并通过Tongyi(model_name="qwen-turbo", dashscope_api_key=api_key)加载模型
- 工具加载：使用load_tools(["serpapi"])加载serpapi工具，该工具已集成在LangChain中

● 智能体类型与初始化



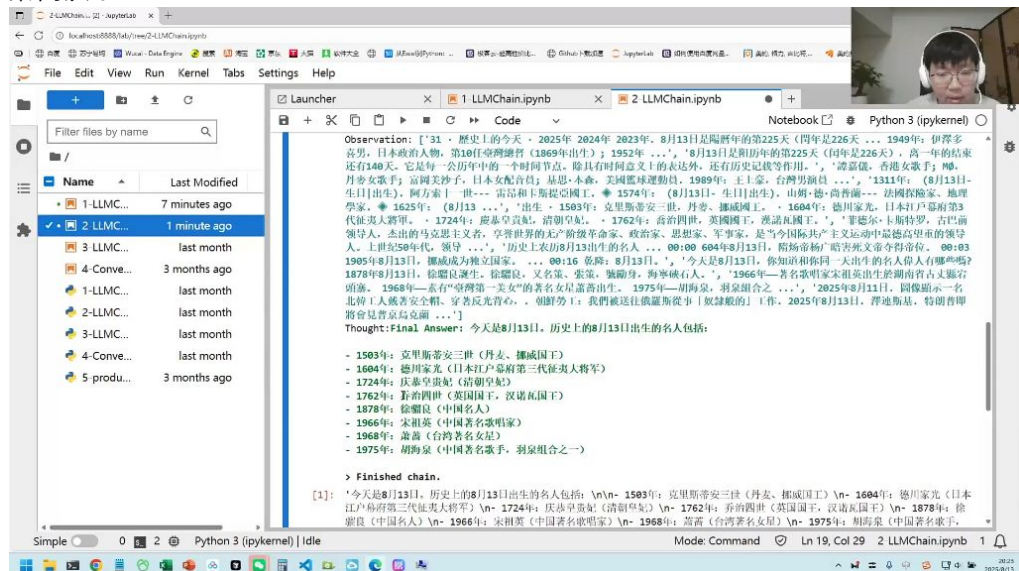
○ 代理类型:

- **zero-shot-react-description**: 根据工具描述和请求内容决定使用哪个工具 (最常用)
- **react-docstore**: 使用ReAct框架与docstore交互, 使用Search和Lookup工具
- **self-ask-with-search**: 只使用Intermediate Answer工具
- **conversational-react-description**: 为会话设计的代理

○ 初始化参数: initialize_agent(tools, llm, agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, verbose=True)

中verbose=True表示打印执行详情

● 实际案例演示



○ 执行流程:

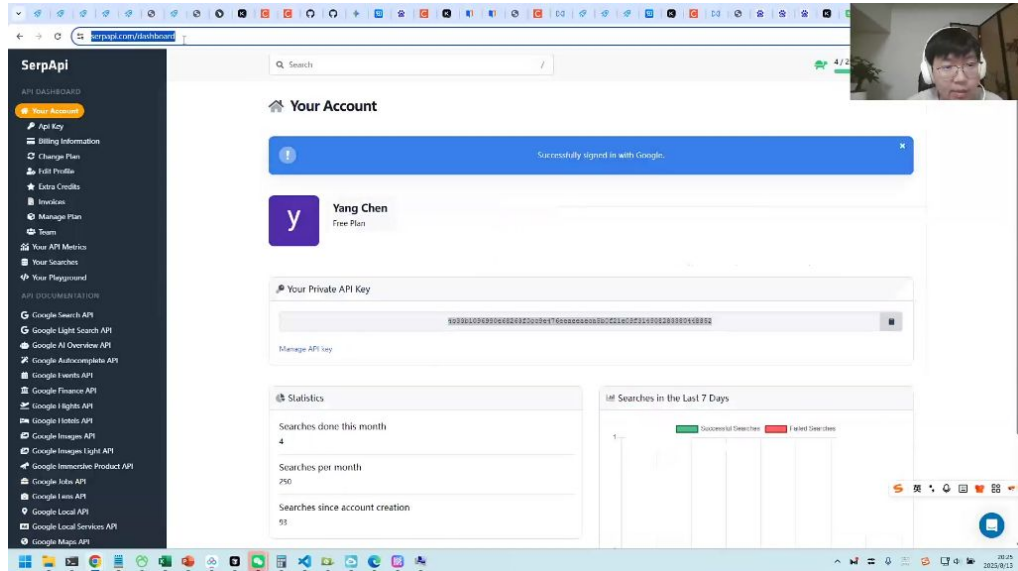
- 询问"今天几月几号, 历史上有哪些名人?"
- AI先思考需要查询当前日期
- 调用serpapi工具查询"今天是几月几号", 返回结果"2025年8月13日"
- 再调用serpapi查询"历史上的8月13日有哪些名人出生"
- 整理返回结果并输出最终答案

○ 返回结果:

- 1503年: 克里斯蒂安三世 (丹麦、挪威国王)

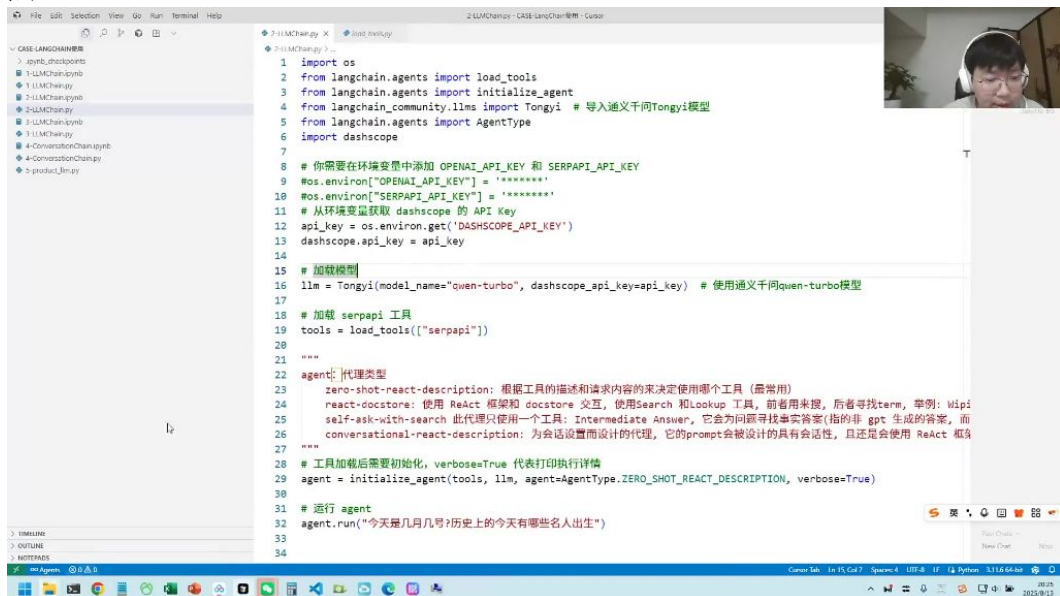
- 1604年：德川家光（日本江户幕府第三代征夷大将军）
- 1724年：庆恭皇贵妃（清朝皇妃）
- 1762年：乔治四世（英国国王）
- 1878年：徐骝良（中国名人）
- 1966年：宋祖英（歌唱家）
- 1968年：萧蔷（台湾女星）
- 1975年：胡海泉（羽泉组合）

● 工具扩展与学习



- 工具查看方法：在IDE中通过"Go to Definition"查看load_tools定义，可找到LangChain集成的各种工具
- SerpAPI获取：访问serpapi.com网站注册账号获取API密钥
- 工具学习建议：通过查看源代码和官方文档学习各工具的使用方法

2. 方法 40:11



- 工具调用原理：通过load_tools加载工具，使用initialize_agent初始化代理，支持多种代理类型
- 代理类型：
 - zero-shot-react-description：根据工具描述和请求内容决定使用哪个工具（最常用）

- react-docstore: 使用ReAct框架与文档库交互
- self-ask-with-search: 仅使用Intermediate Answer工具寻找事实答案
- conversational-react-description: 为会话设计的代理, 使用具有会话性的prompt

3. 应用案例 42:30

1) 例题:温度计算问题

● 题目解析:

- 工具选择: 需要同时调用联网搜索工具 (获取实时温度) 和数学计算工具 (计算四分之一值)
- 实现步骤:
 - 加载serpapi和llm-math工具
 - 初始化代理时指定ZERO_SHOT_REACT_DESCRIPTION类型
 - 传入问题"当前北京是多少华氏度? 这个温度的四分之一是多少?"
- 执行流程:
 - 大模型先调用搜索工具获取当前温度 (23°C)
 - 将摄氏度转换为华氏度 (73.4°F)
 - 调用计算器工具计算四分之一值 (18.35°F)
- 注意事项:
 - 简单计算可直接由大模型完成
 - 复杂运算 (如平方、开根号) 必须使用计算器工具保证准确性
 - 国内用户可替换serpapi为国内类似工具

4. Memory 49:40

Memory



在LangChain中提供了几种短期记忆的方式:

• BufferMemory

将之前的对话完全存储下来, 传给LLM

• BufferWindowMemory

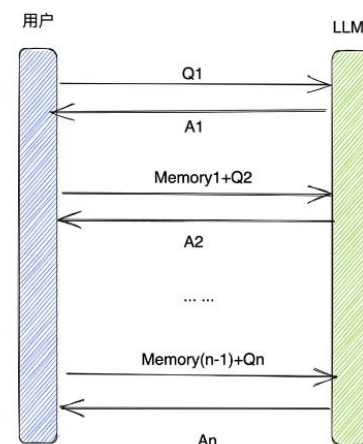
最近的K组对话存储下来, 传给LLM

• ConversationMemory

对对话进行摘要, 将摘要存储在内存中, 相当于将压缩过的历史对话传递给LLM

• VectorStore-backed Memory

将之前所有对话通过向量存储到VectorDB (向量数据库) 中, 每次对话, 会根据用户的输入信息, 匹配向量数据库中最相似的K组对话



- 核心作用: 解决大模型无状态问题, 实现对话上下文记忆
- 四种实现方式:

○ BufferMemory:

- 原理: 完整存储所有历史对话
- 适用场景: 对话量少 (<32k tokens) 的情况

○ BufferWindowMemory:

- 原理: 仅保留最近K轮对话 (可设K=5)
- 缺点: 会丢失早期有价值信息

○ ConversationMemory:

- 原理: 将历史对话压缩为摘要存储
- 优势: 大幅节省token消耗

- 应用：Claude等聊天工具的常用策略
- **VectorStore-backed Memory:**
 - 原理：将对话存入向量数据库，通过相似度检索相关对话
 - 特点：需要额外向量数据库支持，成本较高但效果最好

1) 例题:对话问答 55:14

- 实现演示：
 - 使用ConversationChain建立带记忆的对话链
 - 首轮提问："hi there" → 回应："hello! How can I help you today?"
 - 次轮提问："I'm doing well just have a conversation with AI" → 能结合上文回应
- 技术要点：
 - 记忆隔离：每个AI agent实例独立维护记忆，不同用户对话不会混淆
 - 记忆存储：将用户输入和模型输出自动保存到memory中
 - 记忆使用：在后续对话中将历史记忆作为上下文传入prompt

5. 应用案例 58:11

1) 例题：知识智能客服搭建

Agent设计

AGENT_TMPL = ""按照给定的格式回答以下问题。你可以使用下面这些工具：
{tools}
回答时需要遵循以下用一括起来的格式：

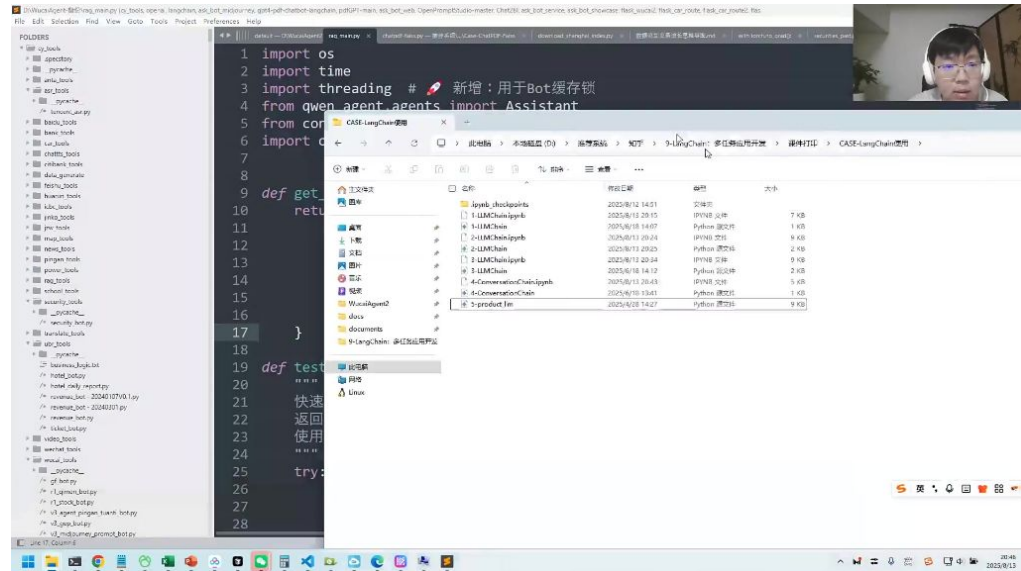
Question: 我需要回答的问题
Thought: 回答这个上述我需要做些什么
Action: "{tool_names}" 中的一个工具名
Action Input: 选择这个工具所需要的输入
Observation: 选择这个工具返回的结果
... (这个思考/行动/行动输入/观察 可以重复N次)
Thought: 我现在知道最终答案
Final Answer: 原始输入问题的最终答案

现在开始回答，记得在给出最终答案前，需要按照指定格式进行一步一步的推理。

Question: {input}
(agent_scratchpad)
"""

这个模板中，最重要的是：
Thought
Action
Action Input
这3个部分是AI在Chain中进行自我思考的过程

- **ReAct模式**：采用Reason-Action思维链模式，通过"思考-行动-观察"循环实现多步推理
- 模板结构：
 - Question：需要回答的问题
 - Thought：思考过程（可重复N次）
 - Action：选择工具名称
 - Action Input：工具输入参数
 - Observation：工具返回结果
 - Final Answer：最终答案

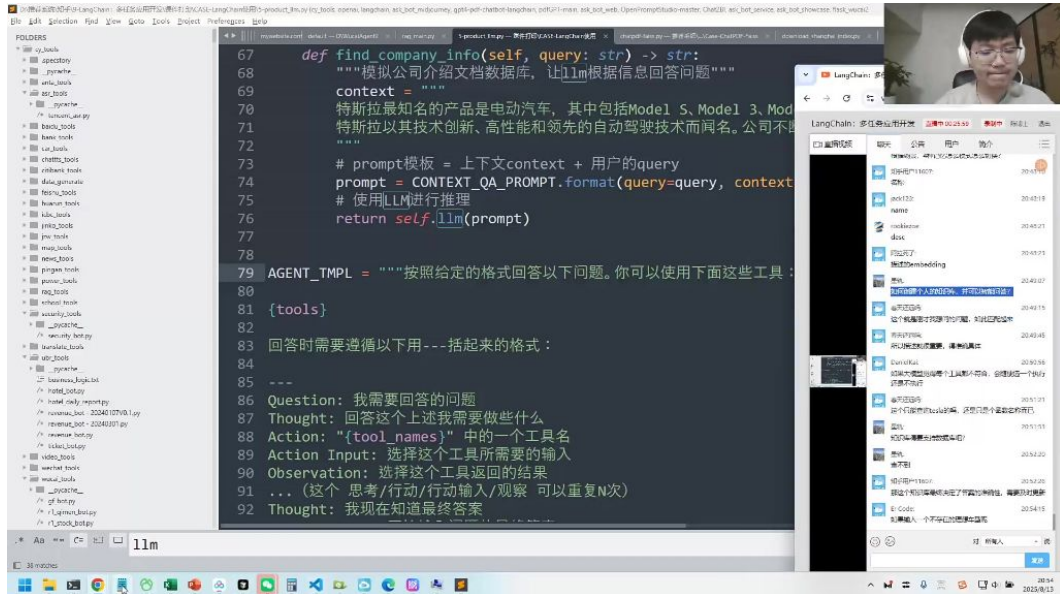


- 工具定义：
 - 产品查询工具：
 - 名称: "查询产品名称"
 - 功能: 通过产品名称查找描述
 - 实现: `find_product_description(product_name)`
 - 示例数据: 包含Model 3/Y/X三款车型参数和定价
 - 公司查询工具：
 - 名称: "公司相关信息"
 - 功能: 回答公司相关问题
 - 实现: `find_company_info(query)`
 - 特点: 直接调用LLM进行推理
- 核心逻辑：
 - 工具选择机制: 根据description字段判断是否调用工具
 - 知识局限性: 仅能回答预设工具支持的问题 (如Model 3/Y/X)
 - 错误处理: 当产品不存在时自动尝试公司查询
 - 动态推理: 通过多次Thought-Action循环解决复杂问题
- 典型执行流程：
 - 用户提问"Model 3怎么样?"
 - Agent识别需要查询产品描述
 - 调用产品查询工具获取参数
 - 返回最终答案: "Model 3具有简洁、动感的外观设计, 定价23.19-33.19万"
- 异常处理演示：
 - 提问"理想L9怎么样?"时:
 - 首次查询返回"没有找到产品"
 - 自动转为查询公司信息
 - 通过LLM推理返回理想汽车品牌信息
 - 最终整合出关于L9的答案
- 实现要点：
 - 提示词设计: 使用变量占位符实现动态交互
 - 工具封装: LangChain的Tool类简化工具集成
 - 知识隔离: 不同用户会话保持独立知识上下文
 - 性能优化: 通过description字段实现精准工具匹配

三、大模型运作机制

- 运作理解：通过具体示例来理解大模型的运作机制是最关键的
- 能力边界：需要明确大模型能回答和不能回答的问题类型

1. 知识库构建方法

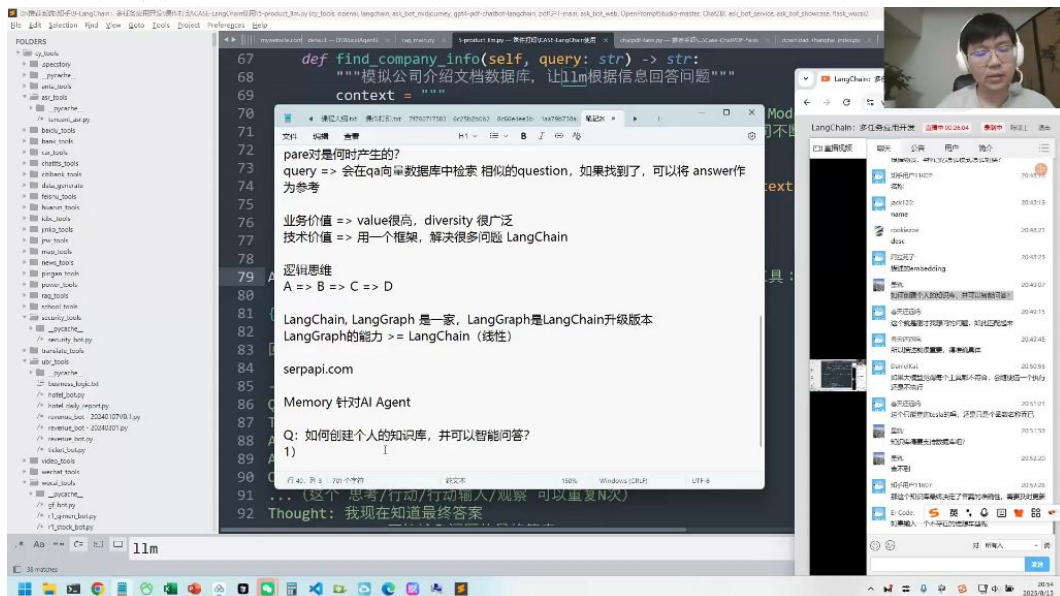


- 实现方法：
- 回答格式：
- 应用要点：
 - 需要建立规范的问答模板
 - 通过工具调用和多次思考迭代来完善答案
 - 明确区分思考过程和最终答案

四、知识库创建01:08:43

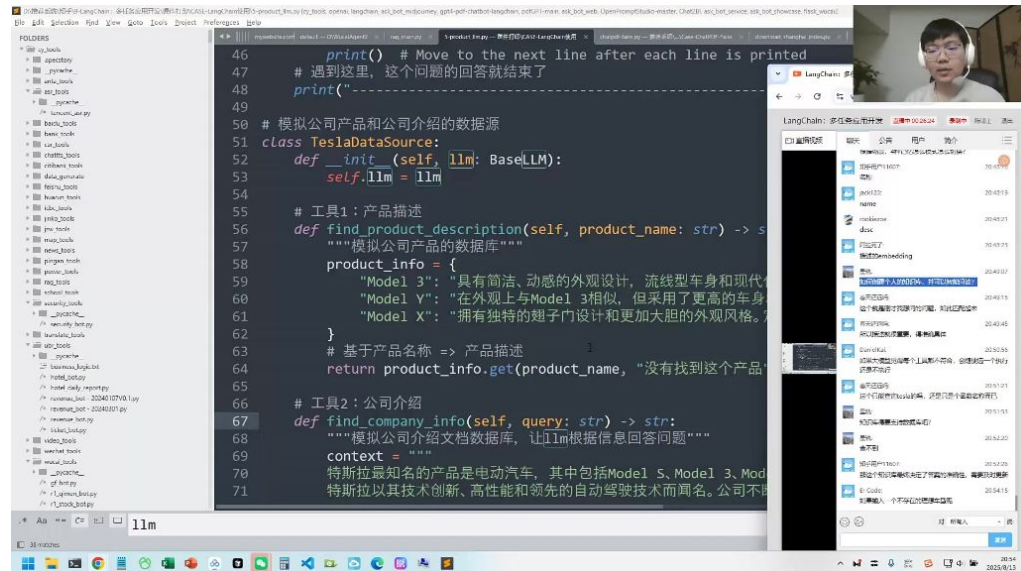
1. 创建知识库并智能问答 01:08:49

1) 方法一：工具方法 01:08:58



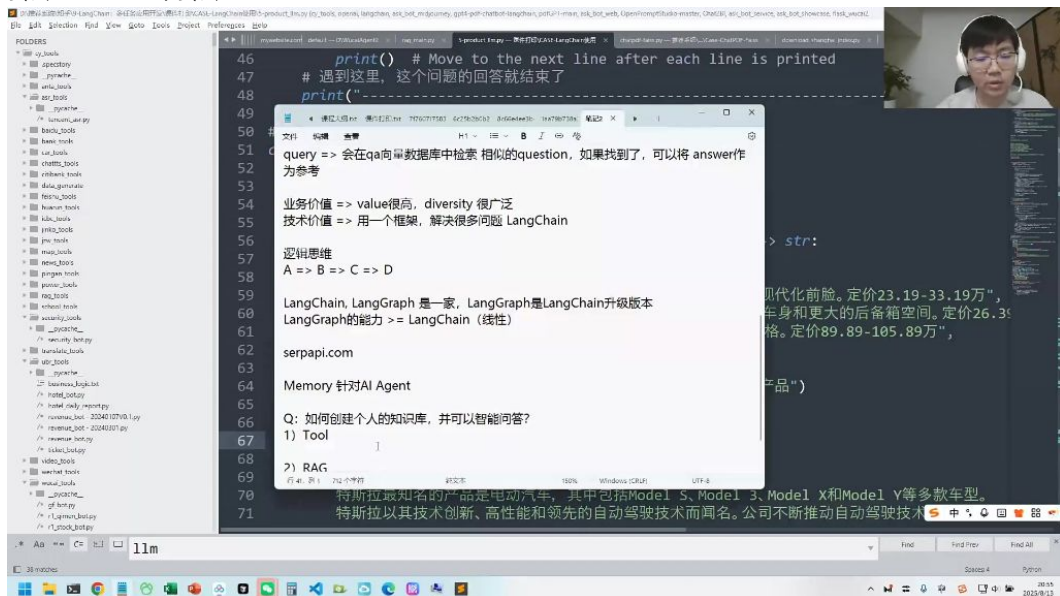
- 实现原理：将知识库封装成工具，通过工具描述与用户query的匹配来调用相应工具
- 业务价值：value很高，diversity很广泛
- 技术价值：用一个框架解决很多问题

- 典型应用：
 - 产品知识库（如Tesla产品描述）
 - 公司知识库（如公司介绍文档）
 - 个人知识库（如产品使用手册、售后服务解决方案）



- 调用机制：
 - 根据query和tool description之间的匹配进行调用
 - 工具描述示例："通过产品名称找到产品描述时用的工具，输入的是产品名称"
 - 工具描述示例："当用户询问公司相关的问题，可以通过这个工具了解公司信息"
- 历史背景：
 - LangChain更早提出Tool调用概念
 - 后来OpenAI将其集成到大模型中
 - 工具调用已成为大模型重要能力

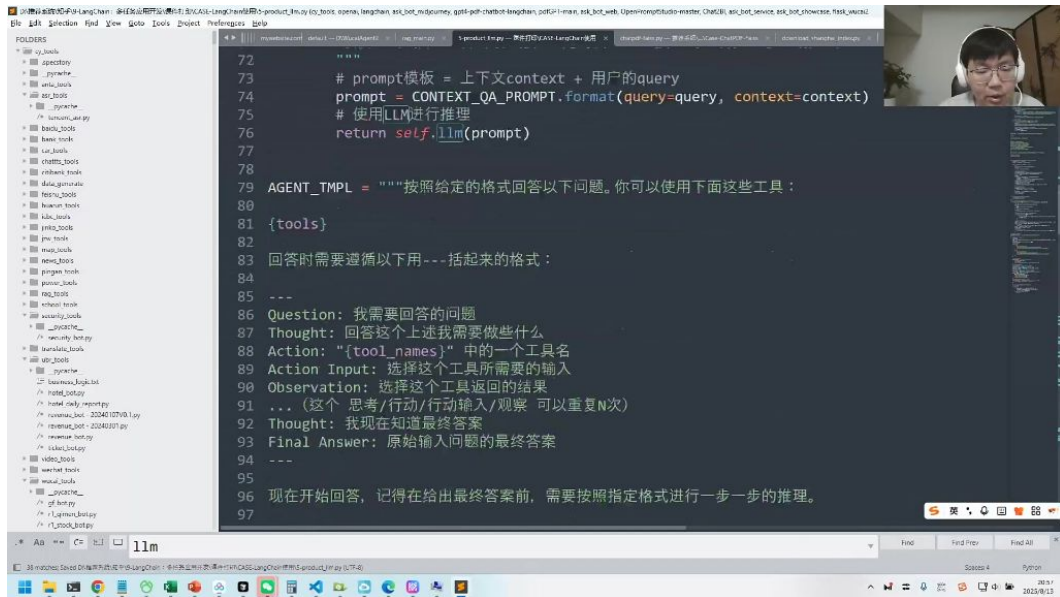
2) 方法二：RAG方法 01:09:37



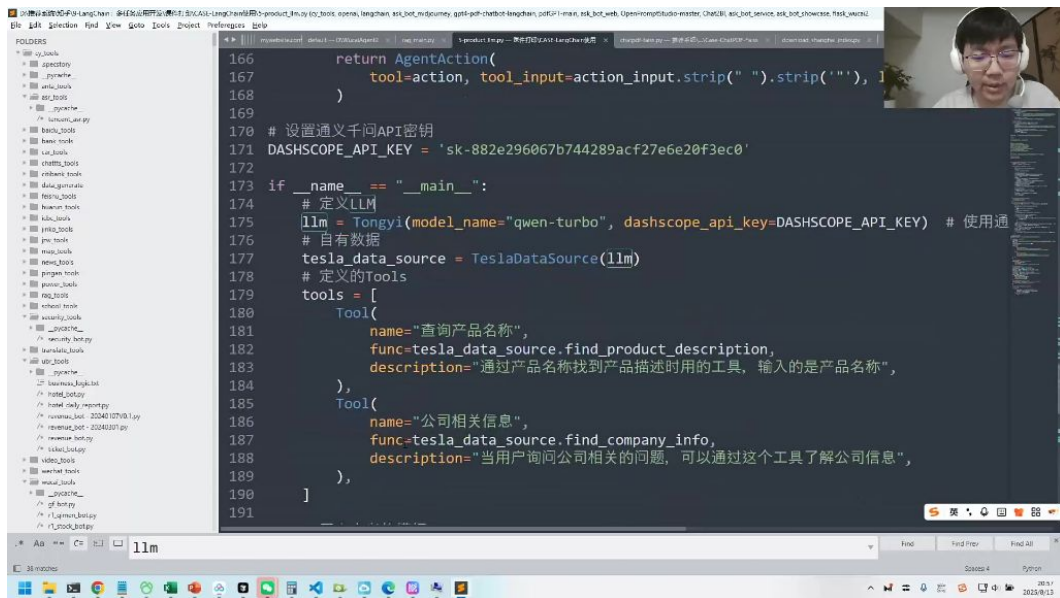
- 核心流程：
 - 将query和知识库中的chunk进行embedding
 - 计算相似度匹配
 - 选出Top5的chunk
 - 给到大模型进行回答

- **特点：**
 - 可视为特殊的工具
 - 更适合知识库问答场景
 - 知识准确性取决于知识库更新频率
- **适用场景：**
 - 智能客服
 - 知识问答
 - 需要基于文档回答的场景

3) 功能插件 01:11:46



- **框架选择：**
 - 取决于框架提供的工具生态
 - 如Coze已有丰富第三方插件生态
 - 私有化部署需考虑数据安全性
 - **工具重要性：**
 - 是智能体的重要组成部分
 - 可扩展大模型能力边界
 - 任何功能都可封装成工具（如看新闻、计算数学题等）
- ### 4) 定义智能体 01:13:00



-
- 执行机制：
 - 遵循Thought => Action => Observation循环
 - 可重复N次直到获得最终答案
 - 使用LLM Chain作为核心组件
- 实现步骤：
 - 定义LLM（如通义千问）
 - 准备数据源（如Tesla数据源）
 - 定义工具集
 - 创建Agent执行器
- 模板示例：

```
1 AGENT_TMPL = ""
2 Question: 需要回答的问题
3 Thought: 思考过程
4 Action: 选择工具
5 Action Input: 工具输入
6 Observation: 工具返回结果
7 ...(可重复多次)
8 Thought: 最终答案
9 Final Answer: 问题答案
10 ""
```

Case: 动手搭建一个本地知识智能客服

```
#定义LLM
llm = Tongyi(model_name="qwen-turbo", dashscope_api_key=DASHSCOPE_API_KEY)

# 自有数据
tesla_data_source = TeslaDataSource(llm)

# 定义的Tools
tools = [
    Tool(
        name="查询产品名称",
        func=tesla_data_source.find_product_description,
        description="通过产品名称找到产品描述时用的工具，输入应该是产品名称",
    ),
    Tool(
        name="公司相关信息",
        func=tesla_data_source.find_company_info,
        description="当用户询问公司相关的问题，可以通过这个工具了解相关信息",
    ),
]

```

让LLM能使用tools，按照规定的prompt执行

在这个prompt中，不断的：

Thought => Action, Action Input => Observation

直到

Thought: 我现在知道最终答案

Final Answer: 原始输入问题的最终答案

- 项目落地：
 - 个人练习可使用Coze等公有云
 - 企业级需考虑私有化部署
 - 数据安全是关键考量因素
- 扩展能力：
 - 可在function call中调用其他API
 - 支持异步操作返回结果
 - 可集成多种第三方服务

2. 知识库的创建与应用 01:17:43

1) dify平台 01:17:47

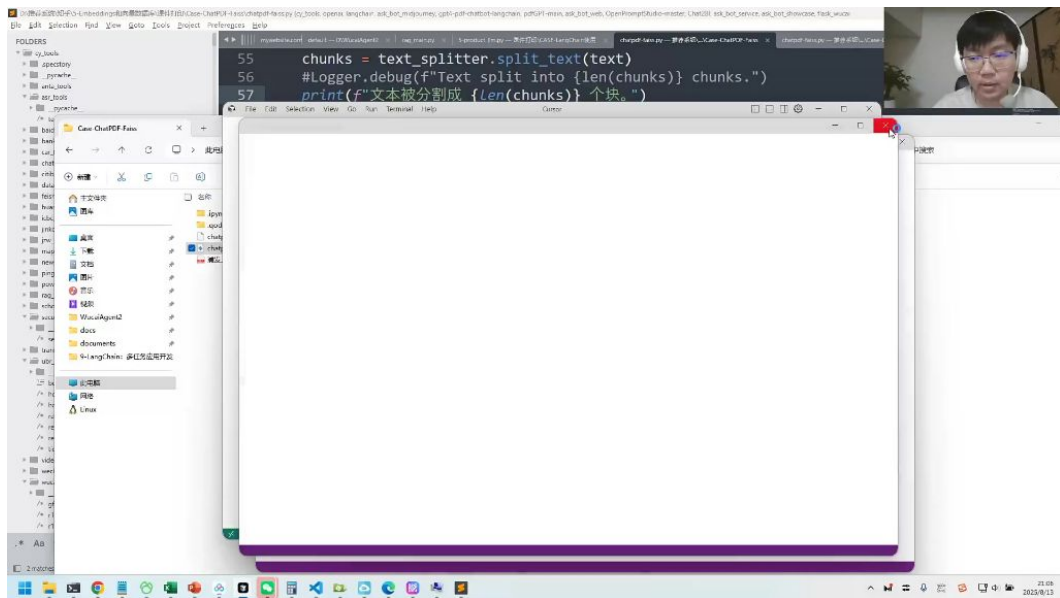
- **低代码特性：**dify是一个低代码平台，可以直接通过Python调用其功能实现知识库应用
- **实现方式：**在代码中通过调用API接口即可完成知识库的搭建和使用

2) 向量数据库 01:18:29

- **核心组件：**
 - **文本切分：**使用RecursiveCharacterTextSplitter按句子进行切分
 - **向量转换：**通过DashScopeEmbeddings将文本转换为向量表示
 - **存储检索：**使用**FAISS**向量数据库存储和检索**向量化**内容
- **实现流程：**
 - 从PDF提取文本和页码信息
 - 将文本切分为chunks
 - 使用**嵌入模型**将chunks转换为向量
 - 构建FAISS知识库并存储向量
 - 保存页码信息用于结果溯源

3. 动手搭建本地知识智能客服 01:24:30

1) 完整实现流程



-
- **数据准备：**
 - 加载PDF文档（示例使用浦发银行客户经理考核办法）
 - 提取文本内容并记录页码信息
- **知识库构建：**
 - 文本切分成5个chunks
 - 使用阿里云DashScope进行向量化
 - 构建FAISS向量数据库
- **问答系统：**
 - 加载QA chain
 - 接收用户query
 - 检索top 2相关结果
 - 将结果和问题一起输入大模型生成答案
- 2) **实际应用案例**
 - **示例问题：** "客户经理被投诉了，投诉一次扣多少分"
 - **系统回答：** 根据知识库内容，回答"每次投诉扣2分"
 - **结果验证：** 答案正确，并能够提供来源页码（第4页和第7页）
 - **成本统计：** 系统会输出处理查询的token使用情况和总成本
- 3) **LangChain优势**
 - **工具集成：** 提供完整的RAG实现工具链
 - **简化开发：** 封装了文本处理、向量化、检索等复杂流程
 - **工厂模式：** 像工厂一样提供各种现成工具组件
 - **效率提升：** 避免从零开发，大幅缩短项目实现周期
- 4. **ReAct范式 01:24:51**
- 1) **例题： ReAct范式应用 01:25:21**

ReAct范式



(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes] ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple _ originally designed to control the Front Row media center program ...

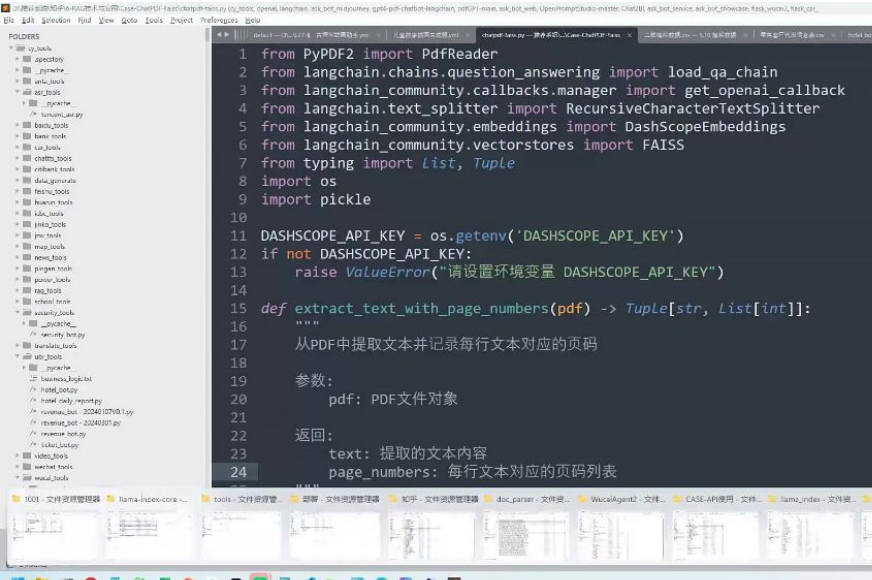
Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys] ✓

- 
- **核心概念:** ReAct = Reason (推理) + Act (行动), 是一种分步推理的交互模式
- **工作流程:**
 - 思考-行动-观察循环: 通过多次"思考需要做什么→执行行动→观察结果"的循环来逐步逼近答案
 - 可解释性: 每个思考步骤和行动都明确记录, 增强过程透明度
- **典型应用:**
 - 苹果遥控器控制设备查询案例:
 - 首先搜索苹果遥控器原始设计用途 (控制Front Row软件)
 - 再搜索Front Row软件的其他控制方式
 - 最终确定键盘功能键是替代控制设备
- **对比传统方法:**
 - 直接回答: 可能给出错误答案 (如iPod)
 - 仅推理不行动: 无法验证假设的正确性
 - 仅行动不推理: 缺乏目标导向性

2) LangChain实现





- **关键组件:**
 - **CustomPromptTemplate:** 定义思考-行动-观察的标准格式模板

- **LLMSingleActionAgent**: 集成LLM链、输出解析器和工具名称
- **执行控制**: 通过stop参数定义终止标记, 限制最大思考次数
- **实现优势**:
 - 通过封装好的ZeroShotReAct框架可大幅减少代码量
 - 支持自定义工具集成 (如搜索工具、安全审核工具等)

5. 总结 01:30:42

1) LangChain中的Chain概念

- **基本组成**: LLM + PromptTemplate + Tools的管道式组合
- **设计要点**:
 - 工具需明确定义名称和输入参数
 - 输出解析器负责标准化响应格式
 - 支持多工具协同工作

2) React模式简化代码 01:31:01

- **优化方法**:
 - 使用预置的ZeroShotReAct框架替代手动实现
 - 标准模板减少重复代码
 - 内置的思考次数限制防止死循环

3) Fewshot和react的区别 01:35:32

- **Fewshot**:
 - 本质: 小样本学习范例
 - 作用: 提供参考案例作为上下文
 - 应用: 主要用于知识库问答中的错题补丁
- **React**:
 - 本质: 推理执行框架
 - 作用: 分步解决问题的工作流程
 - 应用: 复杂问题的探索性求解

4) React模式的应用场景 01:37:50

- **适用情况**:
 - 需要多步推理的复杂问题 (如**投研分析**)
 - 答案需要验证过程的问题
 - 工具调用组合场景
- **限制条件**:
 - 需设置最大思考次数 (通常5-50次)
 - 依赖可靠的工具支持

5) 安全性保障方法 01:38:31

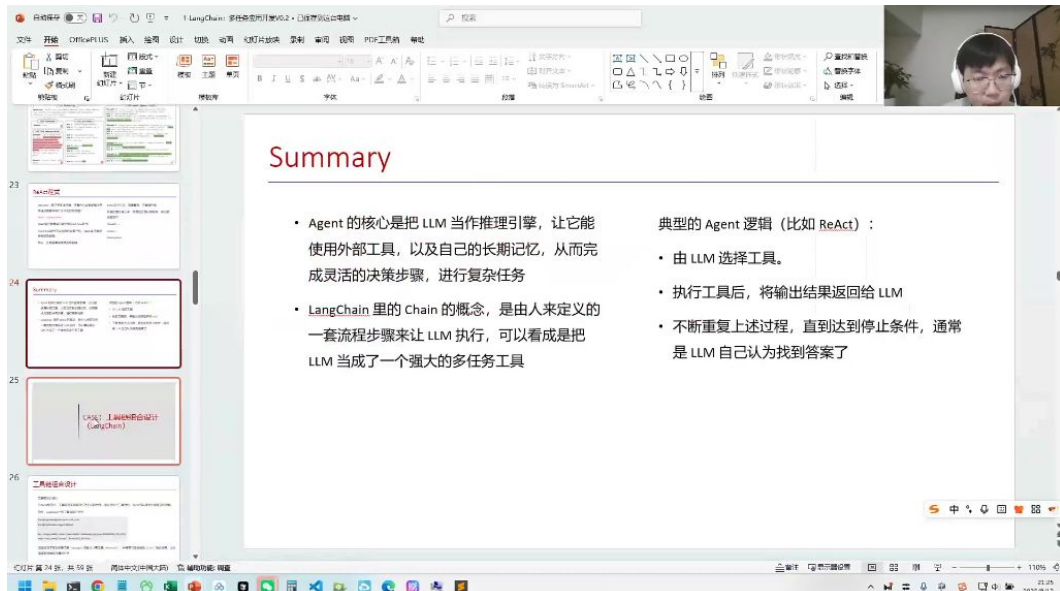
- **主要措施**:
 - 增加安全审核工具 (如关键词过滤)
 - 使用大模型进行内容评估
 - 敏感操作需人工确认
- **实现示例**:
 - 在工具链中加入审核环节
 - 对"造炸弹"类问题自动拦截

6) 工具链组合设计 01:39:39

- **设计原则**:
 - 模块化设计各功能工具
 - 明确工具输入输出规范
 - 支持工具间的数据传递
- **典型组合**:

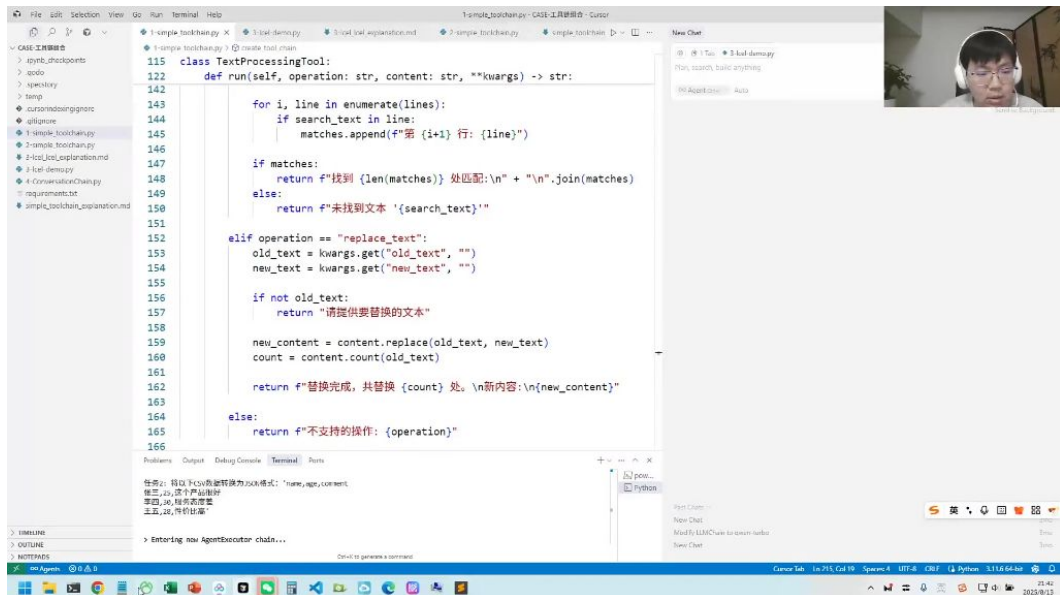
- 搜索工具 + 分析工具 + 审核工具
- 可扩展为多智能体协作系统

五、工具链组合 01:39:52



- 核心原理：将 LLM 作为推理引擎，通过 ReAct 框架实现工具调用和长期记忆的协同工作
- 工作流程：
 - 由 LLM 自主选择工具
 - 执行工具后将结果返回 LLM
 - 循环执行直到满足停止条件
- 与 Chain 的区别：Chain 是预定义流程，而 Agent 是动态决策过程

1. 文本分析工具 01:41:52



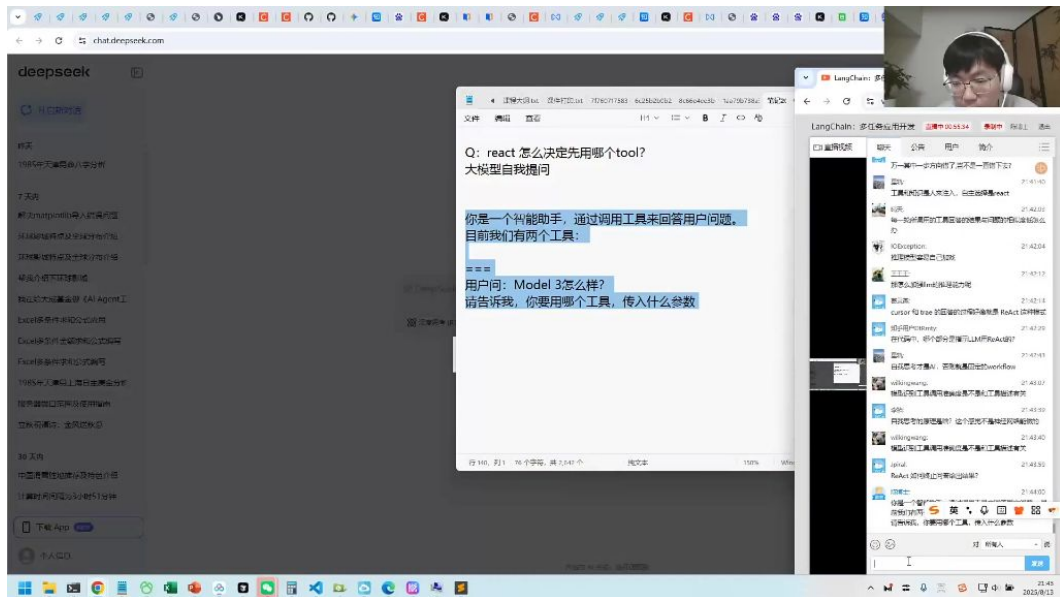
- 功能定义：
 - 名称: "文本分析"
 - 描述: 分析文本内容，提取字符统计和情感倾向
- 参数处理：
 - 输入: 待分析文本字符串
 - 输出: 包含字符数和情感倾向的字典

- 实现要点：
 - 需要明确定义name和description字段
 - 情感分析可采用简单规则（如积极词统计）

2. 数值转换工具 01:42:32

- 功能定义：
 - 名称: "数据转换"
 - 描述: 实现JSON/CSV等格式间的相互转换
- 参数处理：
 - 输入: 原始数据、输入格式、输出格式
 - 输出: 转换后的数据字符串
- 典型应用：
 - CSV转JSON时保持字段对应关系
 - 处理嵌套数据结构时需要特殊处理

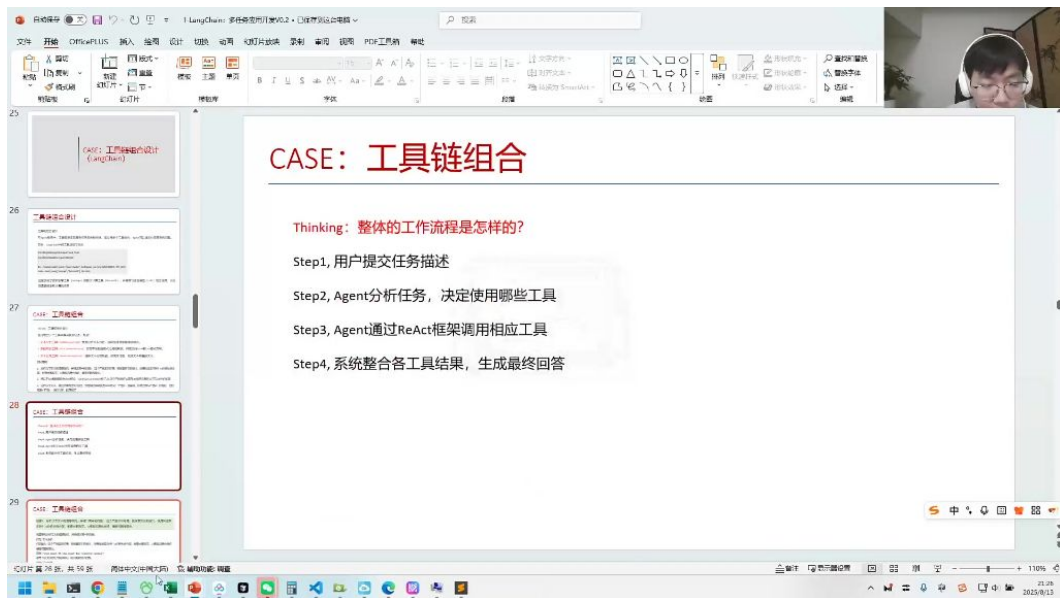
3. 文本处理工具 01:42:57



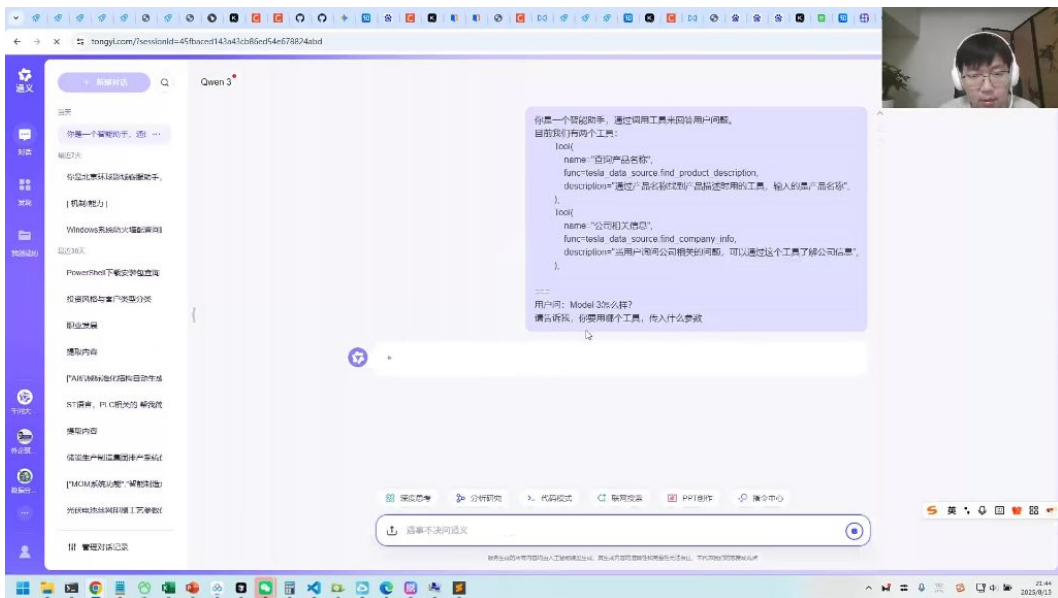
- 功能定义：
 - 名称: "文本处理"
 - 描述: 提供查找/替换/统计等基础功能
- 操作类型：
 - count_lines: 统计行数
 - find_text: 文本查找（返回匹配行号）
 - replace_text: 文本替换（需提供新旧文本）
- 实现细节：
 - 行号从1开始计数
 - 替换操作返回修改次数和新内容
 - 未找到文本时返回明确提示

4. 应用案例 01:44:01

1) 例题:文本分析封装调用



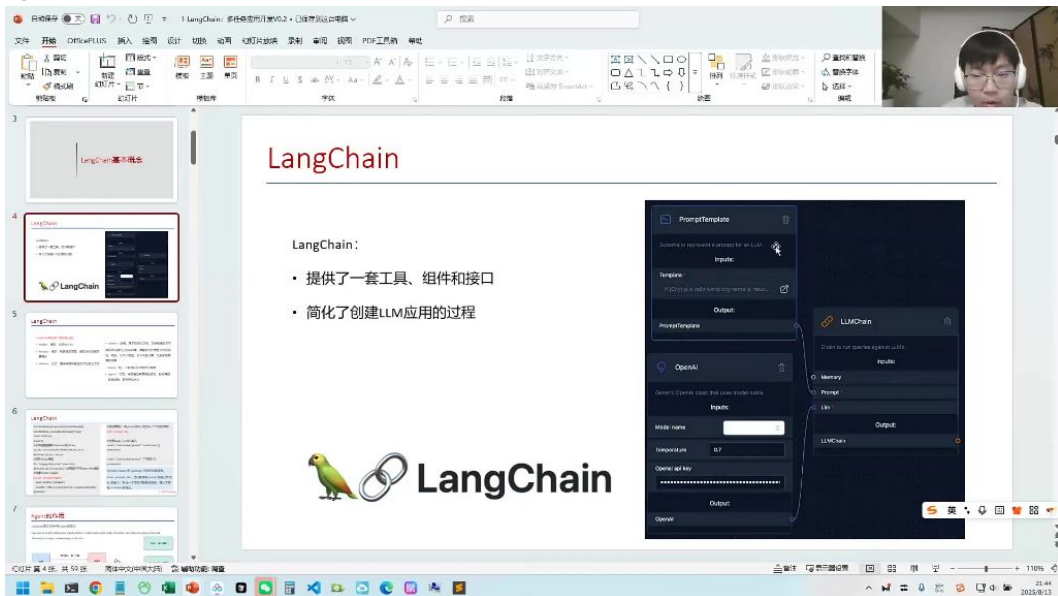
- 执行过程：
 - 用户提问"分析以下文本倾向并统计行数"
 - Agent先调用文本分析工具获取情感倾向
 - 再调用文本处理工具统计行数
 - 整合结果返回最终答案
- 典型输出：
 - 情感倾向：积极
 - 行数统计：3行
- 技术要点：
 - 需要处理工具间的数据传递
 - 注意中间结果的缓存机制
- 2) 例题:格式转换封装调用 01:45:45
 - 执行过程：
 - 用户提供CSV格式数据
 - Agent识别转换需求
 - 调用数据转换工具执行CSV→JSON
 - 返回格式化后的JSON字符串
 - 实现关键：
 - 正确解析CSV表头
 - 处理特殊字符转义
 - 保持数据类型的准确性
- 5. 反应原理与工具拓展 01:47:12
 - 1) 例题:反应原理应用 01:47:22



- 决策机制：
 - 通过prompt工程让LLM自主选择工具
 - 示例问题："Model 3怎么样？"
 - 预期选择："查询产品名称"工具
 - 参数传递：product_name="Model 3"
- 与CoT区别：
 - CoT是预设步骤链（如：先写标题→再写内容→最后结尾）
 - ReAct是**动态决策**过程（自主选择工具和参数）
- 实现原理：
 - 基于对话式自我提问循环
 - 每次迭代包含：思考→行动→观察三个阶段
 - 通过JSON格式解析工具调用指令

六、LangChain总结01:59:26

1. LangChain的基本概念与作用 01:59:29



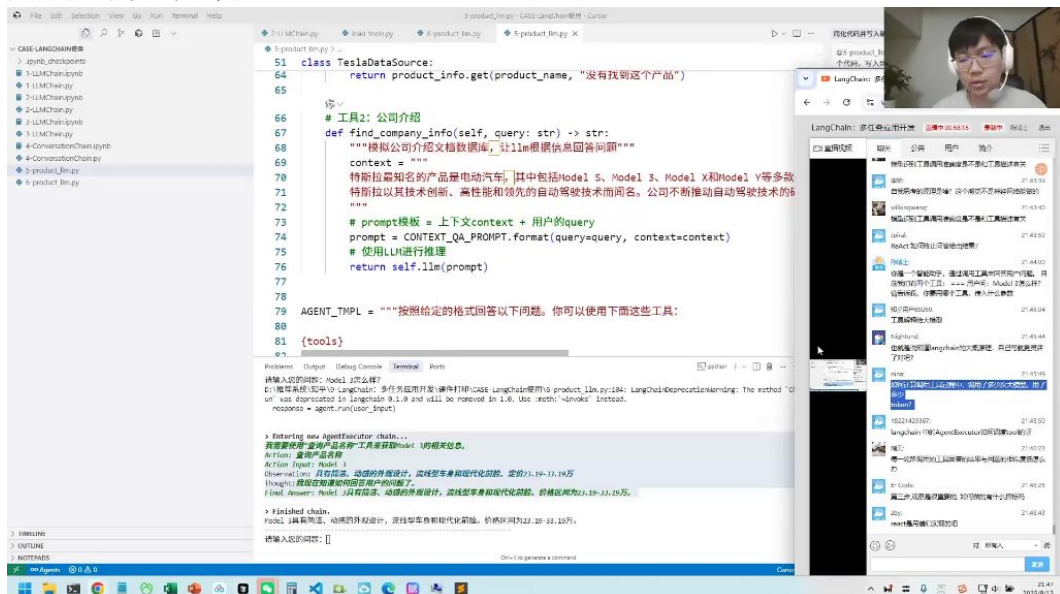
- 核心功能：提供了一套**工具、组件和接口**，简化了创建LLM应用的过程

- **工作流程**：通过定义输入(LangChain inputs)和输出(LLM outputs)模板，实现大模型应用的标准化开发
- **典型应用**：支持多任务开发场景，如产品查询、公司信息检索等业务需求

2. React框架与AI决策 02:00:01

- **自主决策机制**：不同于严格工作流模式，React框架让AI智能体自主决定工具调用顺序
- **优势特点**：通过工具定义和流程编排，实现类似"工厂流水线"的自动化执行
- **应用示例**：当用户询问"Model 3怎么样"时，AI自主选择调用"查询产品名称"工具

3. 工具调用与代码示例 02:00:18



- **工具定义**：通过Tool类封装功能，如产品查询工具包含名称(name)、执行函数(func)和描述(description)
- **参数传递**：工具调用时可传入动态参数，如find_product_description("Model 3")
- **结果处理**：将工具返回结果（如产品描述）作为上下文提供给LLM生成最终回答

4. 开发个性化需求与框架选择 02:00:48

- **开发必要性**：标准化产品无法满足业务快速迭代的个性化需求（如今天加功能明天改逻辑）
- **框架优势**：自主开发可灵活定义工具和流程，如特斯拉案例中的产品/公司信息双工具设计
- **典型场景**：领导预期管理、业务快速响应等需要高度定制化的开发场景

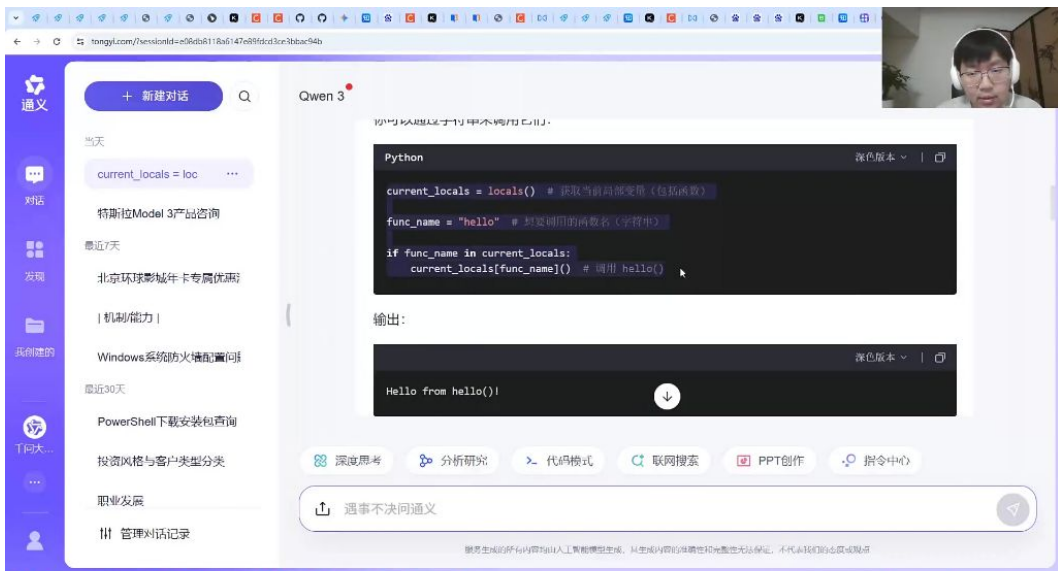
5. 计算工具调用次数与token使用 02:01:32

- **调用统计**：通过跟踪response对象可获取工具调用次数和大模型交互次数
- **Token计算**：response参数包含token消耗量，可通过代码捕获并累加统计
- **调试方法**：建议在中间计算过程输出调用日志，如显示每次工具执行的token用量

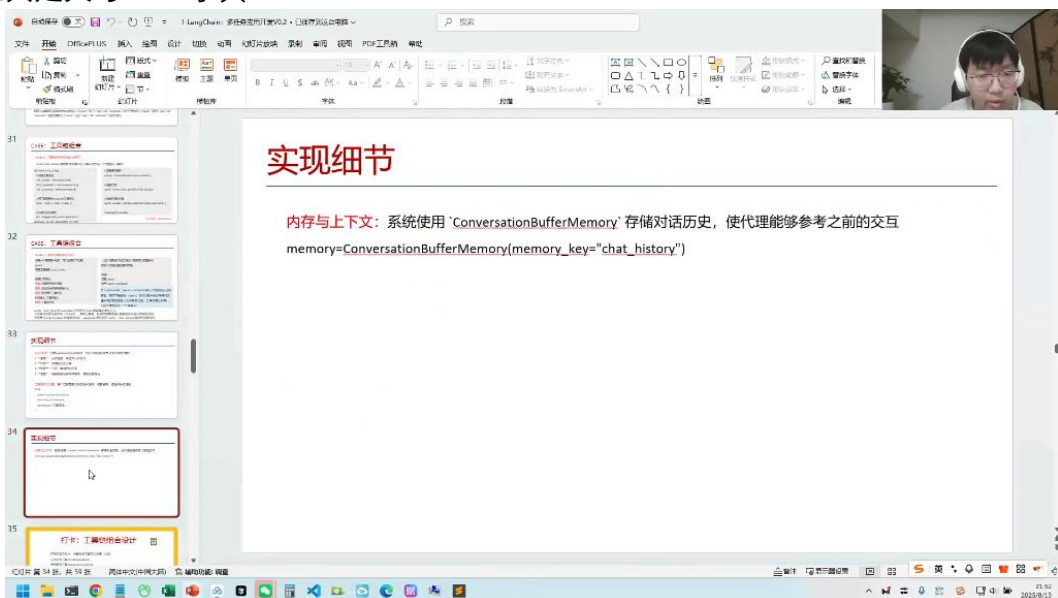
6. 工具调用与反射机制原理 02:02:15

- **反射调用**：通过工具名称字符串反向定位并执行对应函数，如"computer"→compute()
- **实现原理**：利用Python的locals()字典实现名称到函数的映射关系
- **参数传递**：支持动态参数绑定，如compute(args='Model 3')的传参方式

7. 参数传递与函数调用 02:03:57



- **locals应用**: 通过`current_locals = locals()`获取包含所有函数的局部变量字典
 - **动态调用**: 使用`current_locals[func_name]`执行指定名称的函数
 - **参数处理**: 支持带参数调用, 如`greet(name="Model 3")`的传参方式
8. 工具定义与locals字典 02:04:53



- **全局定义**: 在代码全局作用域定义工具函数, 使其能被`locals()`捕获
- **函数注册**: 通过Tool类将功能函数注册到系统, 如特斯拉数据查询工具
- **内存管理**: 使用`ConversationBufferMemory`存储对话历史, 维护上下文一致性

9. 函数调用与locals应用 02:05:55

- **执行流程**: AI输出工具名称→代码通过`locals`查找对应函数→执行并返回结果
- **错误处理**: 需检查`func_name in current_locals`确保函数存在
- **实际案例**: `hello()`函数的动态调用演示, 输出"`Hello from hello()!`"

七、内存与上下文 02:06:54

- **上下文维护**: 使用`ConversationBufferMemory`存储对话历史 (`memory_key="chat_history"`)
- **实现细节**: 通过记忆机制使代理能参考之前的交互, 如保持产品查询的上下文连贯性
- **应用价值**: 在运维诊断等复杂场景中, 历史记录的对问题定位至关重要

八、网络故障诊断Agent 02:07:08

1. 网络故障诊断智能体代码讲解 02:08:05

CASE：搭建故障诊断Agent

帮我搭建一个网络故障诊断类的Agent，工具之间存在串联，第一个工具的输出是第二个工具的输入；如果有10个工具的话，会有很多输入输出的关系。希望Agent，在适当的时候调用适合的工具；

===

可以参考@1-simple_toolchain.py 编写新的Python

```
Let's create the new file 2-network_diagnosis_agent.py.

2-network diagnosis agent.py +281 -1~10 ✓

import os
import json
import re
from typing import List, Union, Dict, Any, Optional

from langchain.agents import Tool, AgentExecutor, create_react_agent
from langchain.prompts import PromptTemplate
from langchain.schema import AgentAction, AgentFinish
from langchain_community.llms import Tongyi
from langchain.memory import ConversationBufferMemory

# --- 环境设置 ---
# 确保设置了您的通义千问 API 密钥
# 您可以通过环境变量 DASHSCOPE_API_KEY 设置，或者直接在这里修改
DASHSCOPE_API_KEY = os.getenv('DASHSCOPE_API_KEY', '您的API密钥')
# 为了方便演示，我们直接在此处硬编码（请注意在生产环境中保护好您的密钥）
DASHSCOPE_API_KEY = 'sk-882e29687b744289acf27e6e20f3ec6' # 1

# --- 自定义网络诊断工具 ---

Qodo Gen: Options | Test this class

class PingTool:
    """执行 Ping 操作以检查与目标的网络连通性。"""
    Qodo Gen: Options | Test this method | 查看
    def __init__(self):
        self.name = "网络连通性检查 (Ping)"
        self.description = "检查本机到指定主机名或 IP 地址的网络"
    Qodo Gen: Options | Test this method | 查看
    def run(self, target: str) -> str:
        """模拟执行 ping 命令。"""
```

```
1 import os
2 import json
3 import re
4 from typing import List, Union, Dict, Any, Optional
5
6 from langchain.agents import Tool, AgentExecutor, create_react_agent
7 from langchain.prompts import PromptTemplate
8 from langchain.schema import AgentAction, AgentFinish
9 from langchain_community.llms import Tongyi
10 from langchain.memory import ConversationBufferMemory
11
12 # --- 环境设置 ---
13 # 确保设置了您的通义千问 API 密钥
14 # 您可以通过环境变量 DASHSCOPE_API_KEY 设置，或者直接在这里修改
15 DASHSCOPE_API_KEY = os.getenv('DASHSCOPE_API_KEY', '您的API')
16 # 为了方便演示，我们直接在此处硬编码（请注意在生产环境中保护好您的密钥）
17 DASHSCOPE_API_KEY = 'sk-882e29687b744289acf27e6e20f3ec6' # 1
18
19 # --- 自定义网络诊断工具 ---
20
21 Qodo Gen: Options | Test this class
22 class PingTool:
23     """执行 Ping 操作以检查与目标的网络连通性。"""
24     Qodo Gen: Options | Test this method | 查看
25     def __init__(self):
26         self.name = "网络连通性检查 (Ping)"
27         self.description = "检查本机到指定主机名或 IP 地址的网络"
28     Qodo Gen: Options | Test this method | 查看
29     def run(self, target: str) -> str:
30         """模拟执行 ping 命令。"""
```

-
- **核心架构**：基于LangChain框架构建，使用通义千问(Tongyi)作为底层大模型，采用 **ReAct(Reasoning and Acting)推理模式**
- **工具串联机制**：第一个工具的输出作为第二个工具的输入，支持10个以上工具的串联调用
- **关键组件**：
 - 环境设置：需配置DASHSCOPE_API_KEY
 - 自定义工具类：如PingTool，包含name、description属性和run方法
 - 工具描述：必须准确描述工具功能，否则会导致大模型误判
 - 会话记忆：通过ConversationBufferMemory实现

CASE：搭建故障诊断Agent

帮我搭建一个网络故障诊断类的Agent，工具之间存在串联，第一个工具的输出是第二个工具的输入；如果有10个工具的话，会有很多输入输出的关系。希望Agent，在适当的时候调用适合的工具；

===

可以参考@1-simple_toolchain.py 编写新的Python

```
Let's create the new file 2-network_diagnosis_agent.py.

2-network diagnosis agent.py +281 -1~10 ✓

import os
import json
import re
from typing import List, Union, Dict, Any, Optional

from langchain.agents import Tool, AgentExecutor, create_react_agent
from langchain.prompts import PromptTemplate
from langchain.schema import AgentAction, AgentFinish
from langchain_community.llms import Tongyi
from langchain.memory import ConversationBufferMemory

# --- 环境设置 ---
# 确保设置了您的通义千问 API 密钥
# 您可以通过环境变量 DASHSCOPE_API_KEY 设置，或者直接在这里修改
DASHSCOPE_API_KEY = os.getenv('DASHSCOPE_API_KEY', '您的API')
# 为了方便演示，我们直接在此处硬编码（请注意在生产环境中保护好您的密钥）
DASHSCOPE_API_KEY = 'sk-882e29687b744289acf27e6e20f3ec6' # 1

# --- 自定义网络诊断工具 ---

Qodo Gen: Options | Test this class

class PingTool:
    """执行 Ping 操作以检查与目标的网络连通性。"""
    Qodo Gen: Options | Test this method | 查看
    def __init__(self):
        self.name = "网络连通性检查 (Ping)"
        self.description = "检查本机到指定主机名或 IP 地址的网络"
    Qodo Gen: Options | Test this method | 查看
    def run(self, target: str) -> str:
        """模拟执行 ping 命令。"""
```

- **实现方法**：
 - 参考simple_toolchain.py编写新文件
 - 每个工具需继承Tool类并实现run方法
 - 通过PromptTemplate控制工具调用逻辑
 - 使用AgentExecutor管理执行流程
- **开发技巧**：
 - 可复用已有代码逻辑，修改业务场景适配

- 工具描述要准确反映实际功能
- 生产环境应避免硬编码API密钥

2. 应用案例 02:13:01

1) 例题:网络故障诊断

CASE：搭建故障诊断Agent



需要检查本机到 www.example.com 的网络连通性。

Action: 网络连通性检查 (Ping)

Action Input: www.example.com--- 模拟执行 Ping: www.example.com ---

Observation: Ping www.example.com 成功: 延迟 59ms。

Thought:网络连通性检查显示可以成功访问 www.example.com, 因此问题可能不在于网络连接。接下来我需要检查 DNS 解析是否正常。

Action: DNS解析查询

Action Input: www.example.com--- 模拟 DNS 查询: www.example.com ---

Observation: DNS 解析 www.example.com 成功: IP 地址是

93.184.216.34

Thought:DNS 解析查询显示 www.example.com 的 IP 地址解析正常。

既然网络连通性和 DNS 解析都没有问题, 可能是其他因素导致浏览器连接超时。

Action: 网络日志分析

Action Input: keywords: 'timeout', time_range: '过去1小时'--- 模拟分析日志: 关键词='keywords: 'timeout', time_range: '过去1小时', 时间范围='过去1小时' ---

Observation: 在过去1小时的日志中找到 3 条与 'keywords: 'timeout', time_range: '过去1小时' 相关的条目:

- [Error] 连接到 10.0.0.88:8080 超时

- [Warning] 对 api.external.com 的请求超时

- [Error] 内部服务通信超时

Thought:网络日志分析显示有几条连接超时的记录, 但它们与外部服务或内部网络相关, 可能并不直接涉及 www.example.com。由于网络连通性和 DNS 解析都没有问题, 浏览器无法访问 www.example.com 的原因可能与其他因素有关, 例如防火墙规则、代理设置或浏览器配置。

Final Answer: 您的网络连通性和 DNS 解析均正常, 但仍然无法通过浏览器访问 www.example.com。建议您.....

- 诊断过程:
 - 检查网络连通性(Ping)
 - 验证DNS解析
 - 分析网络日志
 - 综合判断故障原因
- 典型输出:
 - Action: 工具调用指令
 - Observation: 工具执行结果
 - Thought: 推理过程
 - Final Answer: 最终诊断建议

- 案例解析:
 - 用户问题: 无法访问www.example.com
 - 诊断步骤:
 - Ping测试成功(延迟59ms)

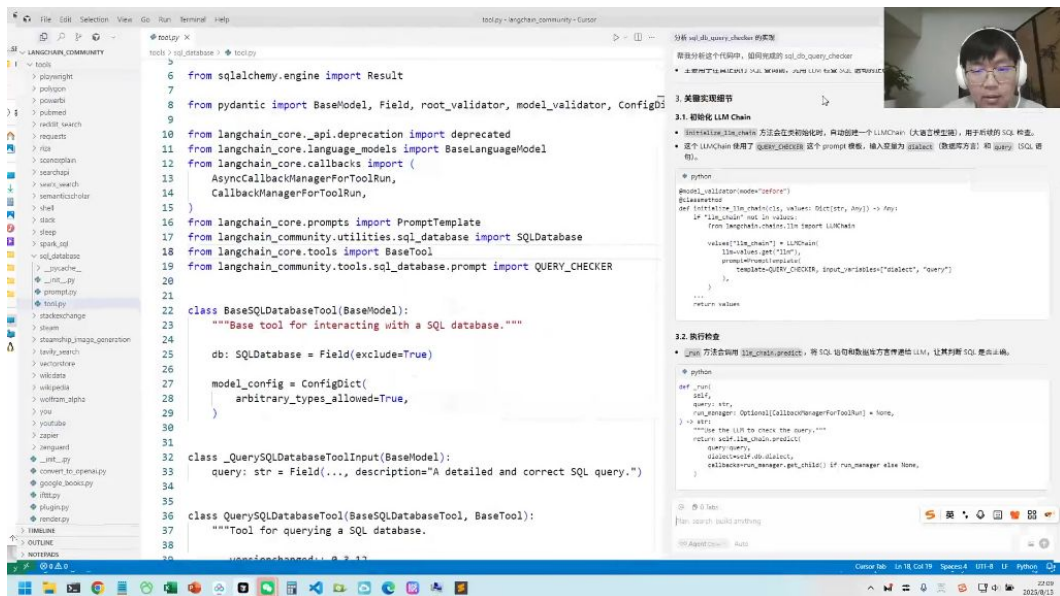
- DNS解析正常(IP:93.184.216.34)
- 日志分析发现3条超时记录
- 最终结论：网络连通性和DNS正常，建议检查防火墙和浏览器配置
- 实现要点：
 - 需自定义故障诊断函数
 - 工具接口需封装成LangChain可调用形式
 - 日志分析应先进行关键词初筛再详细解读



- 常见问题：
 - 输出解析错误：需添加handle_parsing_errors=True参数
 - 工具描述不准确会导致错误诊断
 - 大日志量处理应先初筛再分析
- 优化建议：
 - 对海量日志采用关键词+时间范围初筛
 - 可通过MCP协议访问外部工具
 - 复杂场景应分步骤诊断

九、零样本ReACT特点 02:21:10

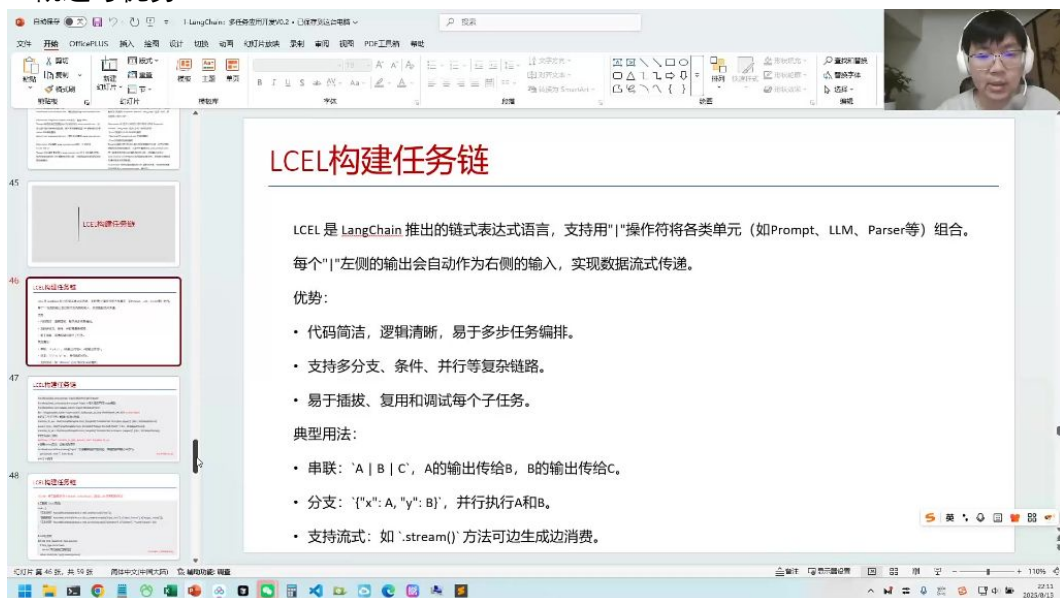
1. CASE: 搭建故障诊断Agent AgentType.ZERO_SHOT_REACT_DESCRIPTION的特点
 - 工具自动读取：自动读取传入的工具(Tool)列表，将每个工具的名称和description自动拼接到系统提示词中
 - 智能决策机制：根据用户输入和工具描述，自动决定调用哪个工具以及如何调用
 - 标准化提示词：自动生成标准英文提示词模板，包含以下关键部分：
 - 工具列表展示："You have access to the following tools:..."
 - 标准响应格式：
2. 关键实现细节：执行检查 02:23:40



-
- **LLM Chain初始化:**
 - 类初始化时自动创建LLMChain(大语言模型链)
 - 使用QUERY_CHECKER模板, 输入变量为dialect(数据库方言)和query(SQL语句)
- **检查执行流程:**
 - 调用llm_chain.predict方法
 - 将SQL语句和数据库方言传递给LLM进行判断
 - 返回检查结果
- **提示词作用:** 主要用于意图理解和参数提取

十、**LCEL** 构建任务链 02:25:36

1. LCEL概述与优势

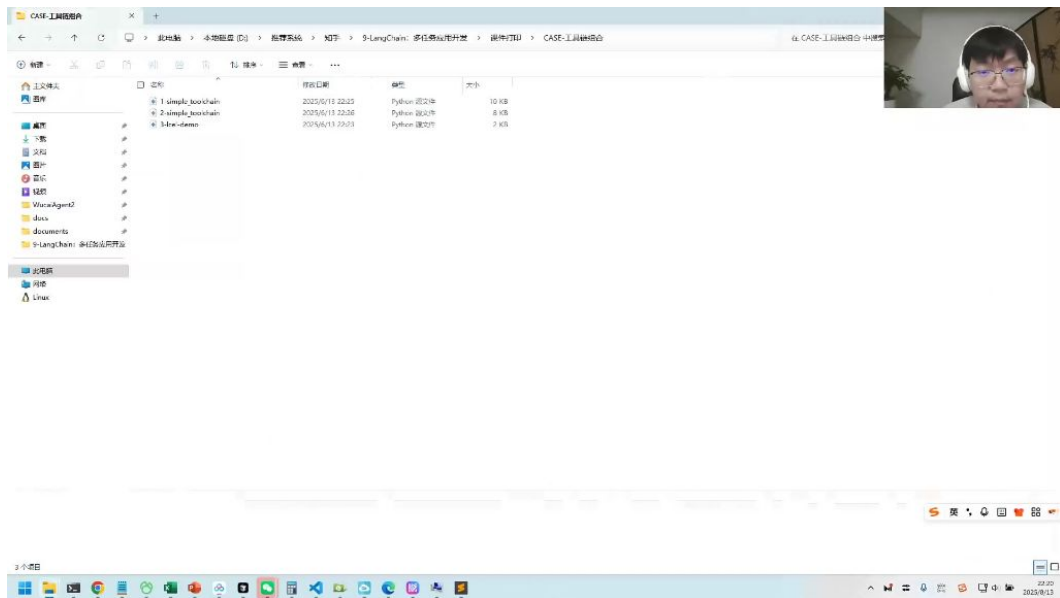


-
- **定义:** LangChain推出的链式表达式语言(LangChainExpression Language)
- **核心操作符:** 使用"|"操作符组合各类单元(Prompt、LLM、Parser等)
- **数据流特性:** 每个"|"左侧输出自动作为右侧输入
- **三大优势:**
 - **代码简洁:** 逻辑清晰, 易于多步任务编排
 - **灵活扩展:** 支持多分支、条件、并行等复杂链路

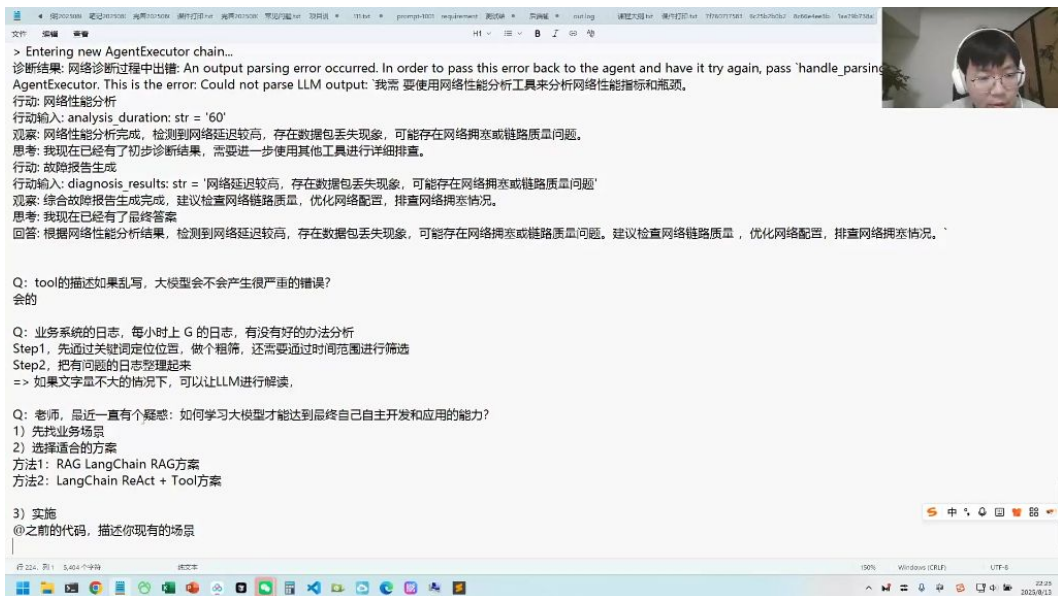
- **模块化:** 易于插拔、复用和调试每个子任务
- 2. LCEL的典型用法 02:26:02
 - **串联模式:** $A | B | C$, A的输出传给B, B的输出传给C
 - **分支模式:** $\{x^n:A, y^n:B\}^n$, 并行执行A和B
 - **流式处理:** 使用stream()方法实现边生成边消费
- 3. LCEL任务示例 02:26:36
 - **三任务工作流:**
 - 翻译到英文任务
 - 大模型处理任务
 - 翻译回中文任务
 - **应用实例:** 用户提问"北京有哪些好吃的地方? 简略回答不超过200字"
 - 流程: 英文回复→处理结果→中文转换
 - **管道符号:** 使用"|"实现数据流传递, 类似Unix管道概念
- 4. LCEL的两种模式 02:30:24
 - **工作流模式:**
 - 特点: 确定流程($A \rightarrow B \rightarrow C$)
 - 适用场景: 固定顺序的任务链
 - **决策模式:**
 - 特点: 动态选择下一步工具
 - 适用场景: 需要智能路由的场景
 - **核心区别:** 前者是预定流程, 后者是自主决策

十一、内容总结 02:31:11

- 1. langChain和langGraph模式 02:31:17
 - **基础架构:** 同属langChain系列技术栈
 - **langChain特点:**
 - 线性任务处理模式
 - 适用于路径清晰的简单工作流 (无复杂循环/条件分支)
 - 典型应用: 工具链自动调度 (如create_react_agent)
 - **langGraph特点:**
 - 支持动态分支管理
 - 可构建自适应系统
 - 适合复杂工作流编排
 - **代码实现:** 两者基础写法相同 (如simple_toolchain.py)
- 2. 自主模式与数据流模式 02:32:09
 - **自主模式:**
 - 代表框架: LangChain Agent
 - 特点: 智能决策+多工具自动调度
 - 适用场景: 需要多轮推理的复杂任务
 - **数据流模式:**
 - 代表技术: LCEL表达式链
 - 特点: 手动编排工具调用顺序
 - 优势: 流程可控, 支持显式条件分支
 - **典型代码:**
 - 自主模式: 1-simple_toolchain.py
 - 数据流模式: 2-simple_toolchain.py
- 3. 开发平台和框架的选择 02:33:19



- 企业偏好：
 - 外企：倾向LangChain/LangGraph
 - 国内企业：多用Coze/Dify等无代码平台
- 无代码平台：
 - 代表产品：Code/Dify
 - 用户群体：业务人员（拖拉拽交互）
 - 优势：降低技术门槛
- 推荐框架：
 - 千问Agent：代码量少（约为LangChain一半），扩展性强
 - 核心优势：内置RAG/MCP等能力，调用简洁
- 4. 案例：工具链组合 02:34:30
 - 典型 workflow：
 - 网络性能分析（检测延迟/丢包）
 - 故障报告生成（输出优化建议）
 - 实现要点：
 - 需要处理LLM输出解析错误（handle_parsing_errors）
 - 工具描述准确性影响决策质量
 - 日志分析技巧：
 - Step1：关键词+时间范围粗筛
 - Step2：问题日志聚类后交LLM解读
- 5. 如何学习大模型开发能力 02:36:08



- 三步学习法：
 - 场景驱动：选择具体业务目标（宜小不宜大）
 - 方案选择：
 - 方案1：千问Agent
 - 方案2：LangChain React+Tool
 - 迭代实施：
 - 基于现有代码修改（需5-10次提示词调优）
 - 需人工校验AI生成逻辑
- 关键提醒：
 - 不要追求完美初版
 - 框架本质是工具组装，简洁性优先
 - 业务理解比代码能力更重要

6. AI Agent对比 02:39:14

AI Agent对比

(1) workflow编排

LangChain: 线性链, 适合固定流程任务 (如文档问答)。

LangGraph: 支持循环、条件边和状态传递, 适合动态调整的复杂逻辑 (如多轮决策)。

Coze: 可视化工作流, 支持嵌套和批处理, 但灵活性较低。

Dify: 基于自然语言定义工作流, 适合API集成和Prompt调优。

(2) 工具调用与扩展性

LangChain/LangGraph: 工具作为链或图的节点, 支持自定义工具和重试逻辑。

Coze: 依赖预置插件生态, 扩展需通过开放平台。

Dify: 支持OpenAPI集成, 适合技术栈复杂的场景。

- 1) workflow编排
- **LangGraph特性**: 支持循环、条件边和状态传递, 适合需要动态调整的复杂逻辑场景 (如多轮决策)
- **Coze特性**: 采用可视化拖拽形式的工作流, 支持嵌套和批处理, 但灵活性相对较低

- **Dify特性**：基于自然语言定义 workflow，特别适合API集成和Prompt调优场景

2) 工具调用与扩展性

- **LangChain/LangGraph**：工具作为链或图的节点，支持完全自定义工具和重试逻辑
- **Coze扩展性**：依赖预置插件生态，扩展需要通过开放平台实现
- **Dify集成能力**：支持OpenAPI集成，适合技术栈复杂的应用场景

7. Coze工作流适配性分析 02:41:29

1) 核心特点

- **可视化平台定位**：本质是拖拉拽的可视化平台，底层开发仍需编写代码
- **文档处理优势**：支持直接拖入文档进行提问，简化文档交互流程
- **多模态实现**：通过插件机制实现多模态功能

2) 私有化部署现状

- **插件成熟度**：目前处于早期阶段（示例：前段时间仅16个插件），多数插件需要注册且不一定可用
- **企业部署建议**：已有企业采用高代码平台进行私有化部署实践
- **开发效率对比**：虽然零码开发写代码速度快，但调试时间可能占开发周期的90%（示例：5分钟编码对应50分钟调试）

3) 适用场景建议

- **快速上手推荐**：建议初学者优先选择Coze进行快速入门
- **复杂场景方案**：推荐使用LangGraph处理需要复杂逻辑的工作流
- **企业级应用**：已有成功案例证明可通过高代码平台实现企业私有化部署

8. 虚拟助教的设计思路 02:42:35

1) 核心框架组成

- **模型选择**：根据场景需求选择合适的大模型，如千问、通义等不同模型
- **Prompt设计**：需明确虚拟助教的指令规则，例如：
 - 仅调用知识库内容回答问题
 - 允许使用的工具列表及调用条件
- **工具集成**：将**知识库检索(RAG)**视为特殊工具，需在description中说明其用途和调用方式

2) 架构模式切换

- **Memory管理**：建议固定使用一种memory模式（如k=5的滑动窗口）
- **对话压缩策略**：当对话历史过长时：
 - 通过大模型对旧对话进行摘要压缩（如压缩至500字）
 - 保留压缩摘要+最新对话内容
- **模式切换触发**：基于对话长度阈值自动触发压缩流程

9. 多个PDF读取的注意事项 02:45:54

- **元数据标注**：必须为每个chunk添加源文件标识，避免内容归属混淆
 - 典型问题：当产品A/B的说明书内容相似时，需明确标注"以下内容属于产品A的XX功能说明"
- **召回策略**：需建立分级检索机制：
 - 基于向量相似度对chunk进行打分排序
 - 对多PDF场景需设置跨文档相关性阈值

10. 长期和短期的memory机制 02:48:36

1) 存储架构

- **长期存储**：使用向量数据库保存完整对话历史
 - 优势：存储容量大，支持全量历史记录
- **短期使用**：通过相似度检索提取相关记忆片段
 - 实现方式：从向量数据库query获取top-k相关对话片段

2) 运行机制

- **更新策略**：定期将新对话内容增量存入向量库
- **检索优化**：为记忆片段添加时间戳等元数据，支持按时间维度筛选

11. 测试用例的编制要求 02:50:01

- **知识库构建**：必须将历史测试用例标准化入库
- **框架选择**：推荐使用千问Agent框架：
 - 优势：集成RAG和工具调用功能完善
 - 适用场景：支持根据故障描述自动生成系统分类测试用例

12. 不同模型的选择 02:51:15

1) 评估方法

- **测试集构建**：准备10+个典型业务问题作为测试基准
- **评分标准**：根据各模型在测试集上的准确率进行选择

2) 多模态处理

- **图文检索**：当涉及以图搜图时：
 - 文本需通过CLIP编码转换为向量
 - 实现图文跨模态相似度计算

13. 打卡练习 02:52:27

1) 实践任务

- **工具链组合**：自主搭建包含ping→traceroute→路由检查的故障诊断工具链
- **场景迁移**：将React模式应用于自身业务场景，构建定制化工具链

2) 进阶挑战

- **复杂场景**：尝试处理超过50节点的大规模拓扑问题
- **知识沉淀**：实现解决方案自动入库的知识管理功能

十二、知识小结

知识点	核心内容	关键工具/框架	应用场景	难度系数
LangChain框架介绍	AI应用开发平台，提供模型管理、提示词管理、记忆管理等功能	LangChain	大模型应用开发	★★
工具调用机制	通过description匹配用户需求，自主选择调用工具	function call	智能客服、运维助手	★★★
四种记忆模式	Buffer Memory/Buffer Window Memory/Conversation Memory/Vector Store Memory	memory组件	多轮对话场景	★★★★
React模式	Reason-Action循环思考模式，分步骤解决问题	zero-shot-react	复杂任务分解	★★★★
RAG实现	文档读取→切分→向量化→相似度检索	vector database	知识库问答	★★★
工具链组合	文本分析/数据转换/文本处理工具组合	serpapi/llm-math	多工具协同任务	★★★★
故障诊断案例	网络连通性检测/DNS解析/端口扫描工具链	network tools	运维排障	★★★★
表达式语法	管道式工作流(chain1 chain2)	LCEL	确定流程任务	★★

框架对比	LangChain(线性)/LangGraph(图)/Coze(低代码)/DeepSeek(高代码)	各开发平台	企业级应用开发	★ ★ ★
业务落地建议	先场景驱动→选择方案→代码实施→逻辑校验	千问agent	实际项目开发	★ ★ ★ ★